Level Up Coding

Member-only story

AI + | Artificial Intelligence + | Data Science ✓ | Machine Learning ✓ | Python ✓

Jev Clearly Explained

Teaching a Model to Answer With a Number Instead of a Paragraph



Fareed Khan Follow 22 min read · Sep 21, 2026

1K 16 6

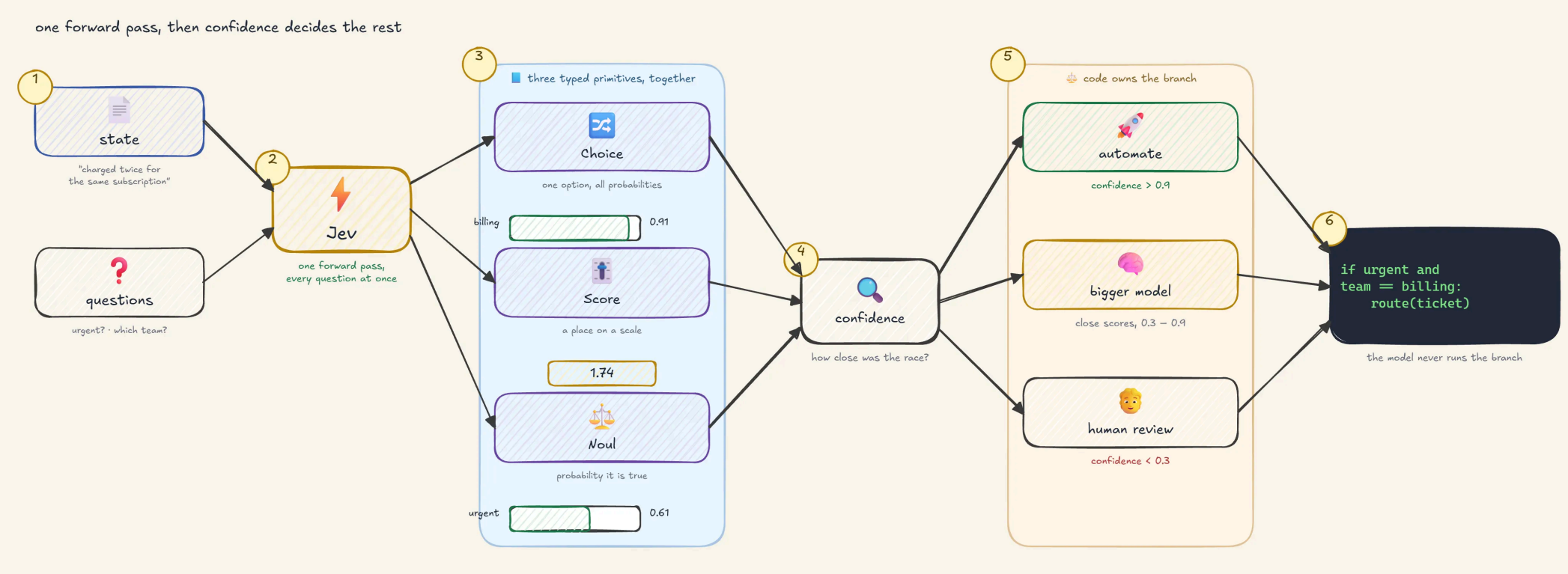
Read this story for free: link

Say I have an app that needs to know whether an invoice looks like fraud. I could hand a language model the invoice and the vendor history and ask it what it thinks.

It writes back something like “**Based on the line items and vendor history, this invoice appears to be legitimate,**” generated one token at a time. My code then has to read it, guess at what it means, and hope the wording never breaks my parser.

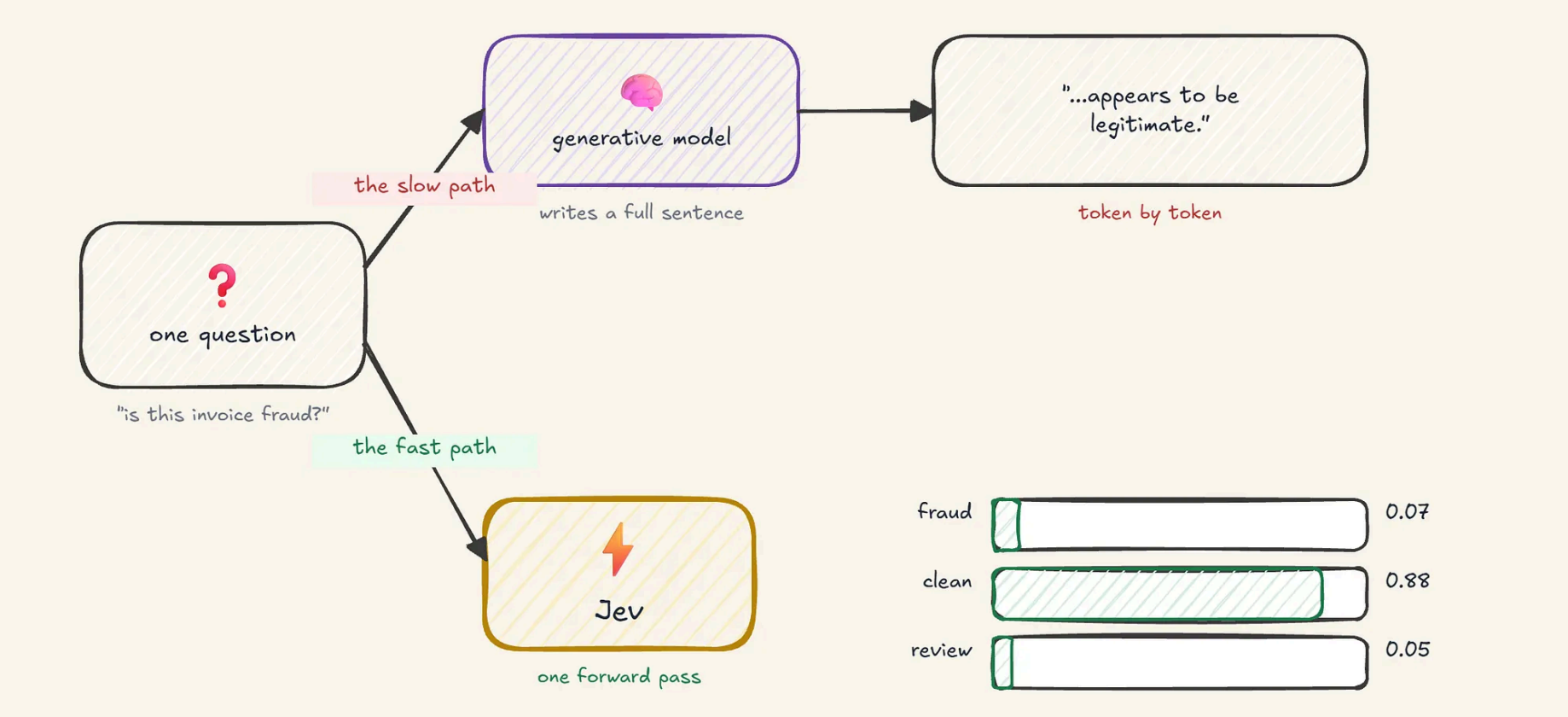
Or I could ask a different kind of model to just score the three outcomes I care about, fraud, clean, review, and get three numbers back in one pass. No sentence to parse, just `fraud 0.07, clean 0.88, review 0.05.`

Here is that whole idea in one picture, worked through on a real ticket, ending at the exact line of code that runs.



State and questions go into Jev, every question is scored in the same pass, a confidence threshold decides the branch, and ordinary code runs it, never the model (Created by Fareed Khan)

Zooming into just the first step of that picture, the choice between writing an answer and scoring one, looks like this.



Two ways to answer the same question, one writes a sentence, the other returns three numbers (Created by Fareed Khan)

That second model is Jev, a decision engine TypeSafe AI released this year. It cannot hold a conversation, write a paragraph, or generate a line of code.

TypeSafe calls it a “System One model,” the fast, instinctive kind of thinking, not the slow, deliberate kind. It answers the small, instant judgments our code needs constantly, never anything that requires thinking deeply.

I want to understand how this works instead of taking the announcement at face value. So we are going to build the same trick ourselves, look at the mechanism that lets a model skip writing text, and check where the published numbers hold up.

The Problem We Keep Paying Full Price For

Most of what software needs from a language model is not a paragraph, it is a judgment. Is this support ticket urgent? Which model should handle this request? Is this shell command dangerous? Does this passage answer the question? None of that needs prose, just one of a handful of known answers.

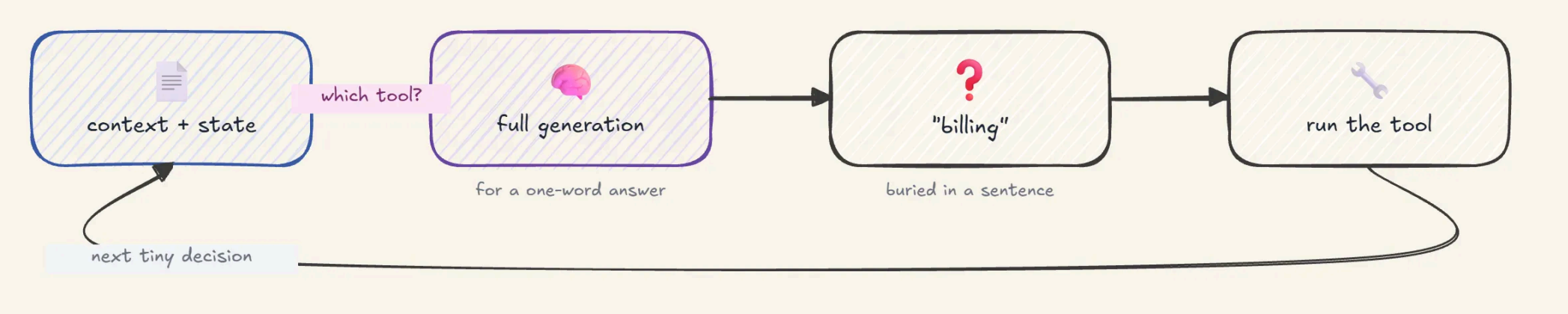
The trouble is we reach for the same tool no matter the question, sending it

Medium Search Play Games Write

parsing that back into the label we wanted in the first place. Put that inside an agent loop and the cost compounds fast, an agent calls a model constantly for exactly this kind of small decision.

```
while not task_done:
    action = llm(context)          # full generation, just to pick a tool
    result = run_tool(action)
    context += result
    task_done = llm(context)       # full generation again, just to check "done
```

Every one of those calls pays the price of a generative model, token by token, even though the answer is a handful of bits.



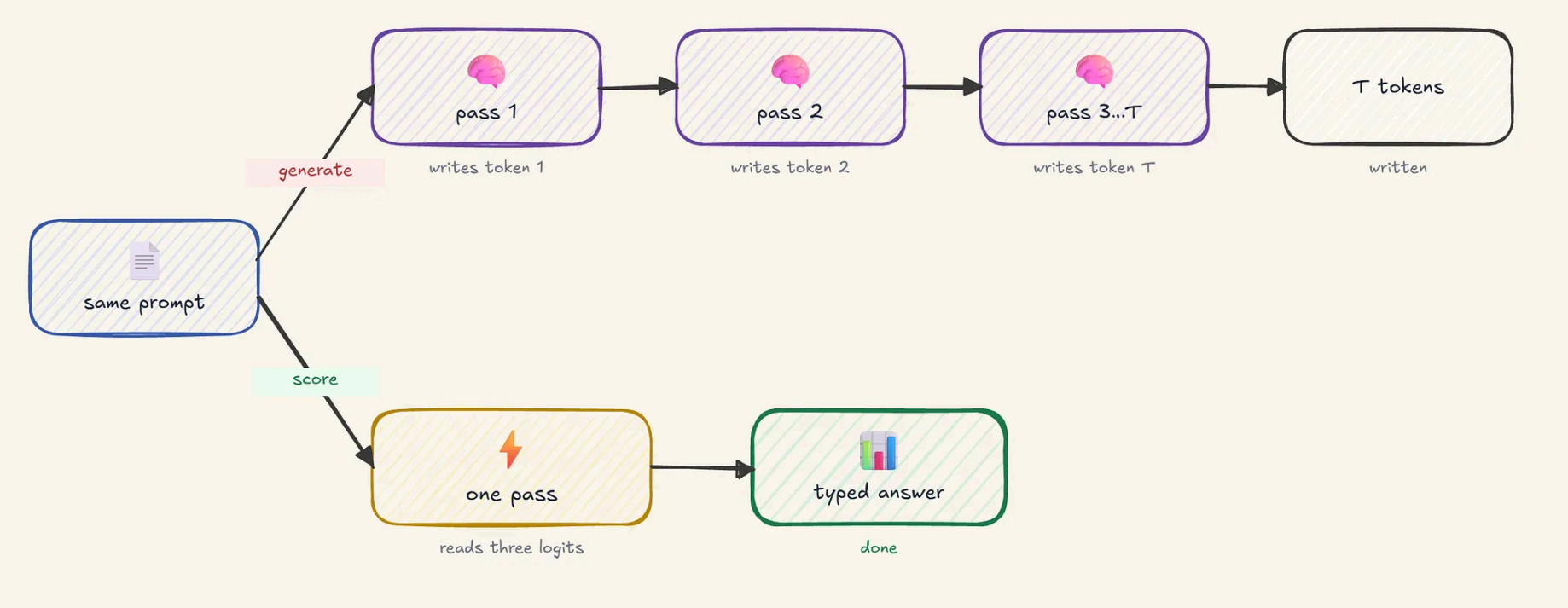
A naive agent loop calls a full generative model for every tiny decision it makes (Created by Fareed Khan)

The gap between the two comes down to a literal count of how many times the model has to run.

$$\text{cost}_{\text{generate}} = T \text{ sequential forward passes}, \qquad \text{cost}_{\text{score}} = 1 \text{ forward pass}$$

Writing T tokens costs T passes through the model, one at a time, a bounded answer never needs more than one

Writing T tokens costs T sequential passes through the model, a bounded answer never needs more than one
(Created by Fareed Khan)



The same prompt, one branch runs the model T times to write T tokens, the other runs it once to read three logits
(Created by Fareed Khan)

Every pass in the generate branch depends on the token the previous pass just wrote, so they run one at a time. The score branch has nothing to wait on.

Jev's bet is that language generation is the wrong interface once code already knows the possible answers. If I can enumerate the outcomes ahead of time, I need a model that scores, not one that writes.

What Jev Is

The shortest accurate description of Jev is a semantic decision engine. I send it **state**, the current situation, and one or more **questions** about that state. It hands back a typed answer with a probability, never a sentence.

That is the picture at the top of this post. State and questions go in, every question gets evaluated in the same pass, and the probability that comes back is what my own code branches on, not the model.

Every question declares its answer shape up front. Jev supports three of these primitives.



Choice picks one option with probabilities, Score places the input on a scale, Noul answers yes or no with a probability
(Created by Fareed Khan)

Choice picks one option out of a list I define and returns a probability for every option, not just the winner. **Score** places the input on an ordered scale I define, so I get back something like a 1.74 on a missed-to-resolved scale. **Noul** answers a yes or no question by returning the probability that it is true, TypeSafe's own name for this boolean-flavored primitive.

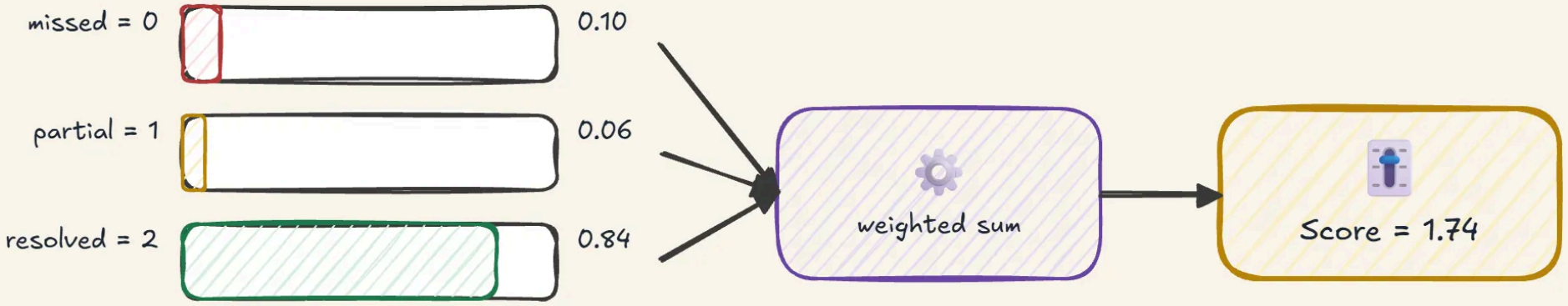
What matters more than the name is that every answer comes back as a number between 0 and 1 my code can act on directly.

That 1.74 is worth a second look, it is not a rounding artifact. Score computes the probability-weighted average across every rung on the scale, not just the top pick.

$$\text{Score} = \sum_k k \cdot P(\text{level} = k)$$

An ordered scale's value is a probability-weighted average across every rung, which is why it lands on 1.74, not a whole number

The Score primitive is a probability-weighted average across every rung on the scale, not a single pick (Created by Fareed Khan)



Missed, partial, and resolved each carry a probability, and the weighted sum of those three numbers is what lands at 1.74 (Created by Fareed Khan)

If the model were certain the ticket was resolved, the score would land close to 2.0. Landing at 1.74 instead tells me it leans strongly toward resolved but keeps real weight on partial, information a single label would throw away.

Here is what a request looks like, using the support ticket I will carry through the rest of this post.

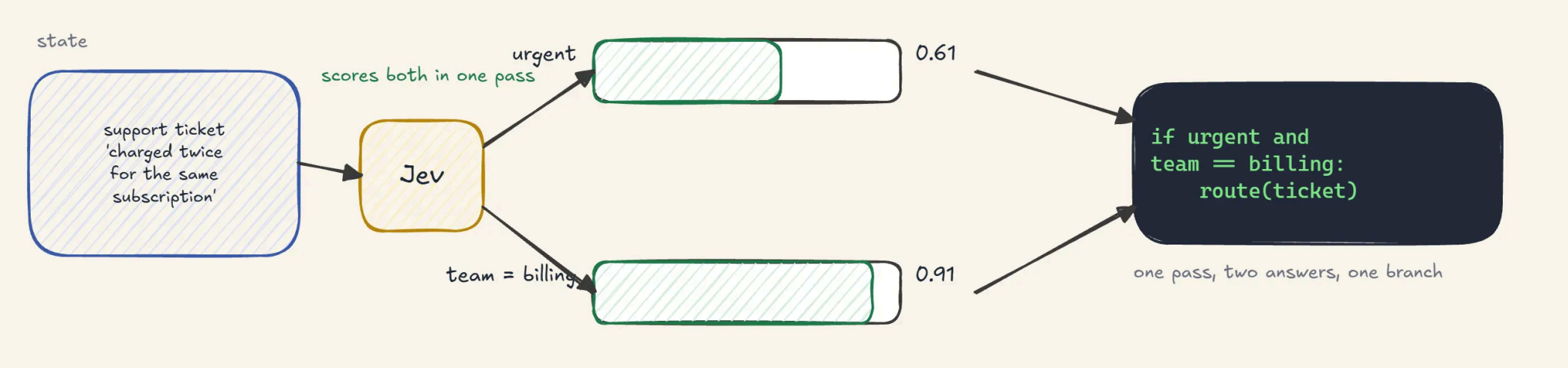
```
{
  "model": "jev-latest",
  "state": "I was charged twice for the same subscription.",
  "questions": {
    "urgent": {
      "type": "noul",
      "instructions": "Does this need attention right now?"
    },
    "team": {
      "type": "choice",
      "instructions": "Which team should own this ticket?",
      "criteria": {
        "billing": "Charges, invoices, refunds",
        "technical": "Product bugs and errors",
        "account": "Login and access problems"
      }
    }
```

```
  }
}
```

Every question gets evaluated in the same pass. The response holds no prose at all, just the labels and the numbers behind them.

```
{
  "urgent": {"probability": 0.61},
  "team": {
    "choice": "billing",
    "probabilities": {"billing": 0.91, "technical": 0.06, "account": 0.03}
  }
}
```

Here is that exact ticket traced end to end, one forward pass turning into two branch-ready numbers at once.

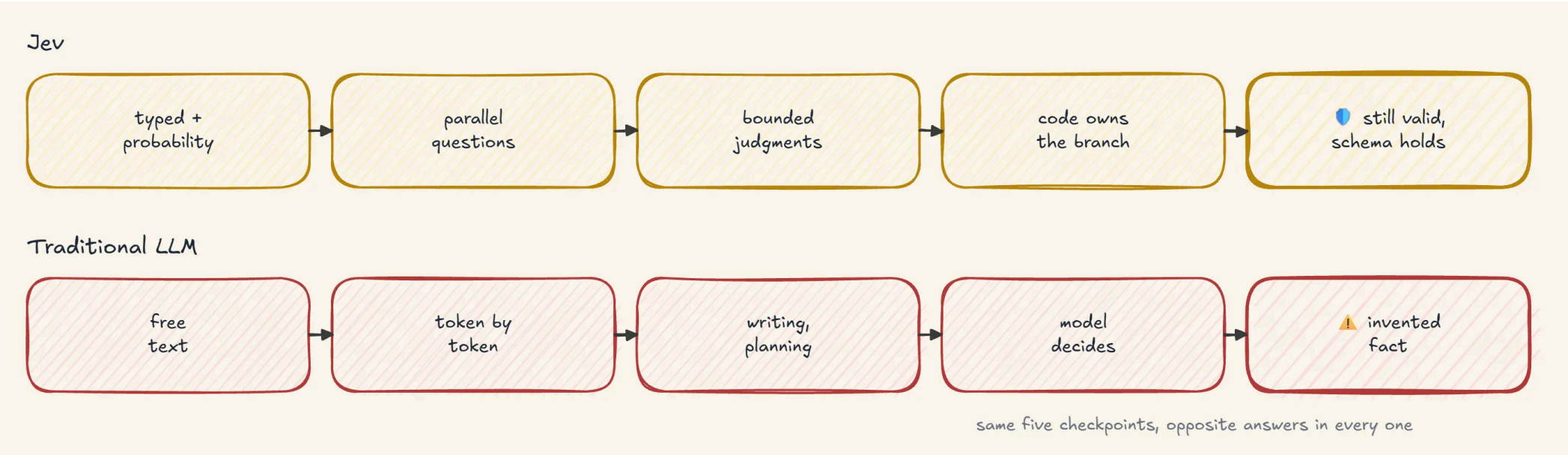


The support ticket goes into Jev once, urgent and team are both answered from that same pass, and the branch fires directly off the two probabilities (Created by Fareed Khan)

My code reads that response directly. There is no fourth team for the model to invent, `team` can only ever be one of the three names I gave it.

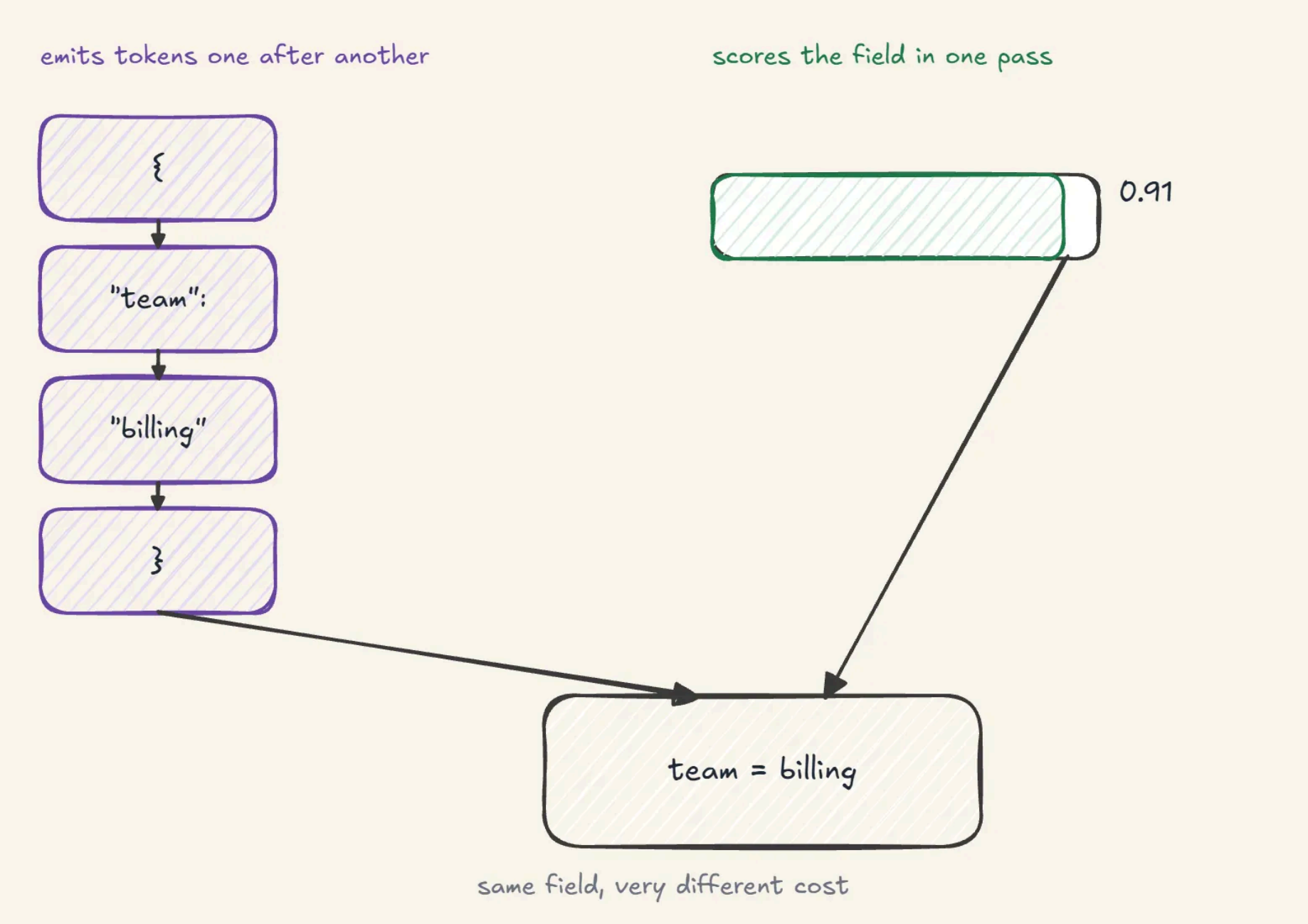
This is different from asking a model for structured output. A structured-output call still decodes one token at a time, the schema only constrains the shape those tokens take. Jev's scoring path never enters that decode loop, it reads the answer off a single forward pass.

Lined up across the same five checkpoints, output, sampling, best fit, control, failure mode, the two approaches diverge from the very first row and never converge again.



Traditional LLM and Jev compared across output, sampling, best fit, control, and failure mode, the same five checkpoints, opposite answers in every row (Created by Fareed Khan)

Structured output and fixed-answer scoring split the same way. One still runs the full autoregressive loop underneath its schema, the other exits before that loop ever starts.


emits tokens one after another
scores the field in one pass
{
"team":
"billing"
}
0.91
team = billing
same field, very different cost

Structured output stays autoregressive under its schema, fixed-answer scoring reads its answer before the decode loop begins (Created by Fareed Khan)

Where That Probability Comes From

To see why Jev can skip the decode loop, it helps to remember what a normal generation step does. A tokenizer turns my prompt into token IDs, the model produces one vector for the next position, holding one score, a logit, for every token in the vocabulary, tens of thousands of numbers.

Turning a logit into a probability means running softmax against every other logit in that whole vocabulary.

$$P(\text{token} = v) = \frac{e^{\ell_v}}{\sum_{u \in V} e^{\ell_u}}, \qquad |V| \approx 150{,}000$$

Ordinary generation normalizes one logit against the whole vocabulary, every single time

Softmax over the whole vocabulary, one normalization shared across tens of thousands of candidates (Created by Fareed Khan)

v there is the entire vocabulary. Ordinary generation does not even run that plain version, it divides every logit by a temperature first, which sharpens or flattens the distribution before picking a token.

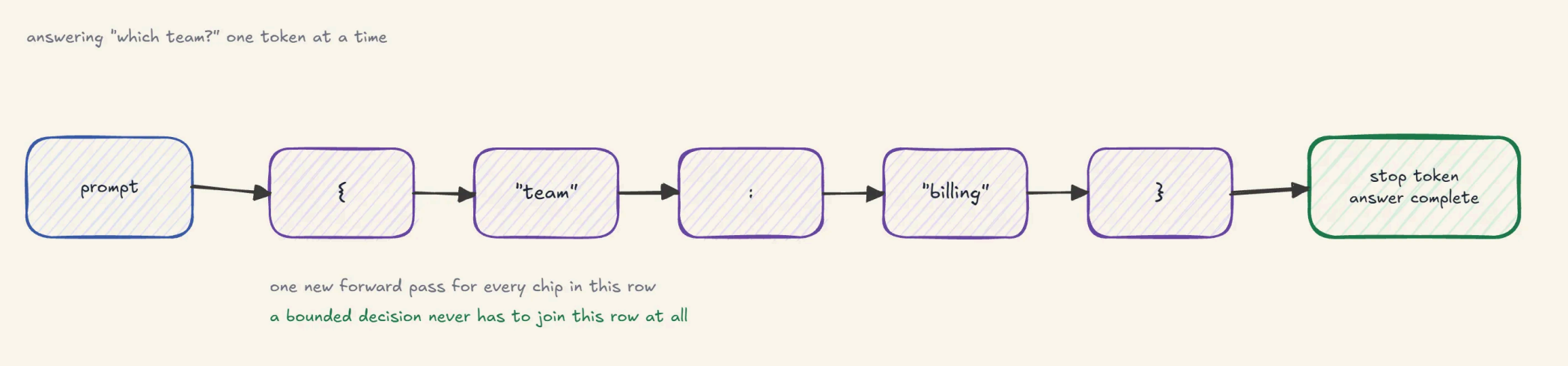
$$P(\text{token} = v) = \frac{e^{\ell_v/\tau}}{\sum_{u \in V} e^{\ell_u/\tau}}$$

Ordinary decoding divides every logit by a temperature before softmax, restricted scoring needs no such knob

Prompt and generated tokens feed one forward pass, decoding rules turn logits into one appended token, and the loop repeats until a stop token or limit ends it (Created by Fareed Khan)

That knob exists because generation has to decide how much randomness to inject into an open-ended answer. Restricted scoring never asks that question, so it just reports the ratio the raw logits already imply.

That whole cycle, forward pass, decoding rules, one appended token, repeats until a stop token or a length limit ends it.

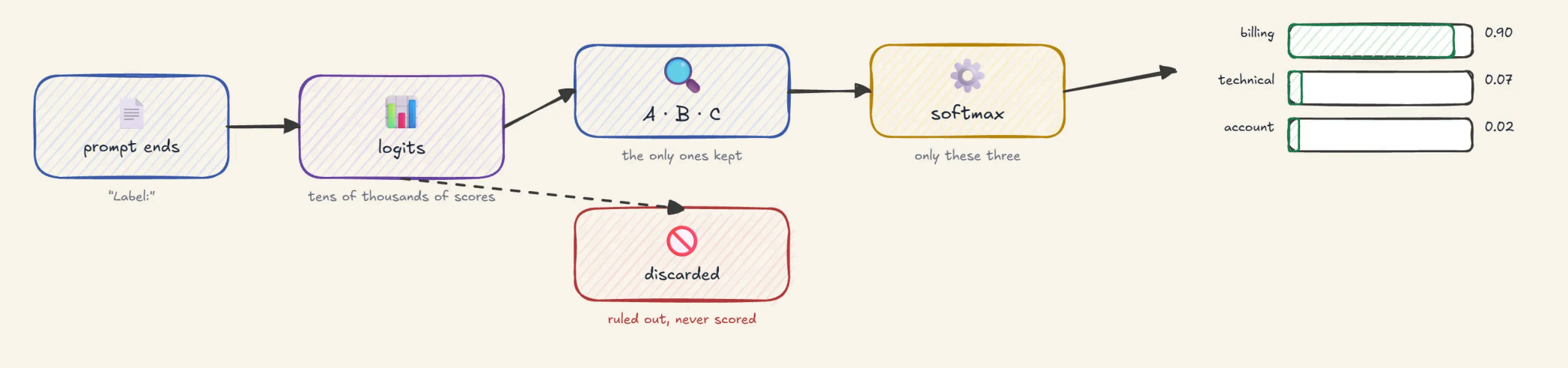


Temperature scales every logit before the softmax, sharpening the distribution below 1 and flattening it above 1
(Created by Fareed Khan)

But if I only care about a bounded decision, I only need the very first vector, and only three of its entries.

Say I label my three answers `A`, `B`, `C` in the prompt, and end it right where the model would normally start writing the label. The model still produces its usual vector for that position, holding the logits for `A`, `B`, `C`, and every other token.

I ignore all of the others, keep only the three I care about, and apply softmax to just those three.



The whole vocabulary produces a score, everything except the three declared labels gets discarded, and only those three ever reach the softmax (Created by Fareed Khan)

The equation itself barely changes, only the denominator does.

$$P(\text{label}_i \mid \{A, B, C\}) = \frac{e^{\ell_i}}{e^{\ell_A} + e^{\ell_B} + e^{\ell_C}}, \qquad i \in \{A, B, C\}$$

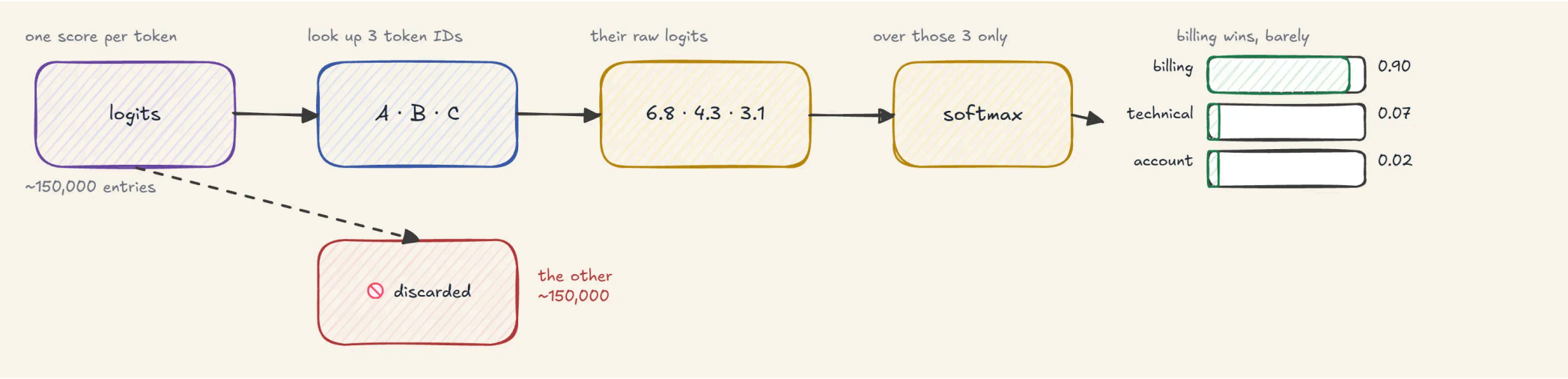
Jev's trick: the same logits, normalized over only the three declared labels, nothing else survives

The same softmax, denominator restricted to exactly the three declared labels (Created by Fareed Khan)

Say those three logits come out as `A = 6.8, B = 4.3, C = 3.1`, running that restricted softmax gives roughly `billing 0.90, technical 0.07,` account 0.02`.

I am not asking whether billing has 90 percent probability across the entire vocabulary. I am asking how the model splits its preference among exactly the three choices my code allowed, swapping a sum over 150,000 terms for a sum over 3.

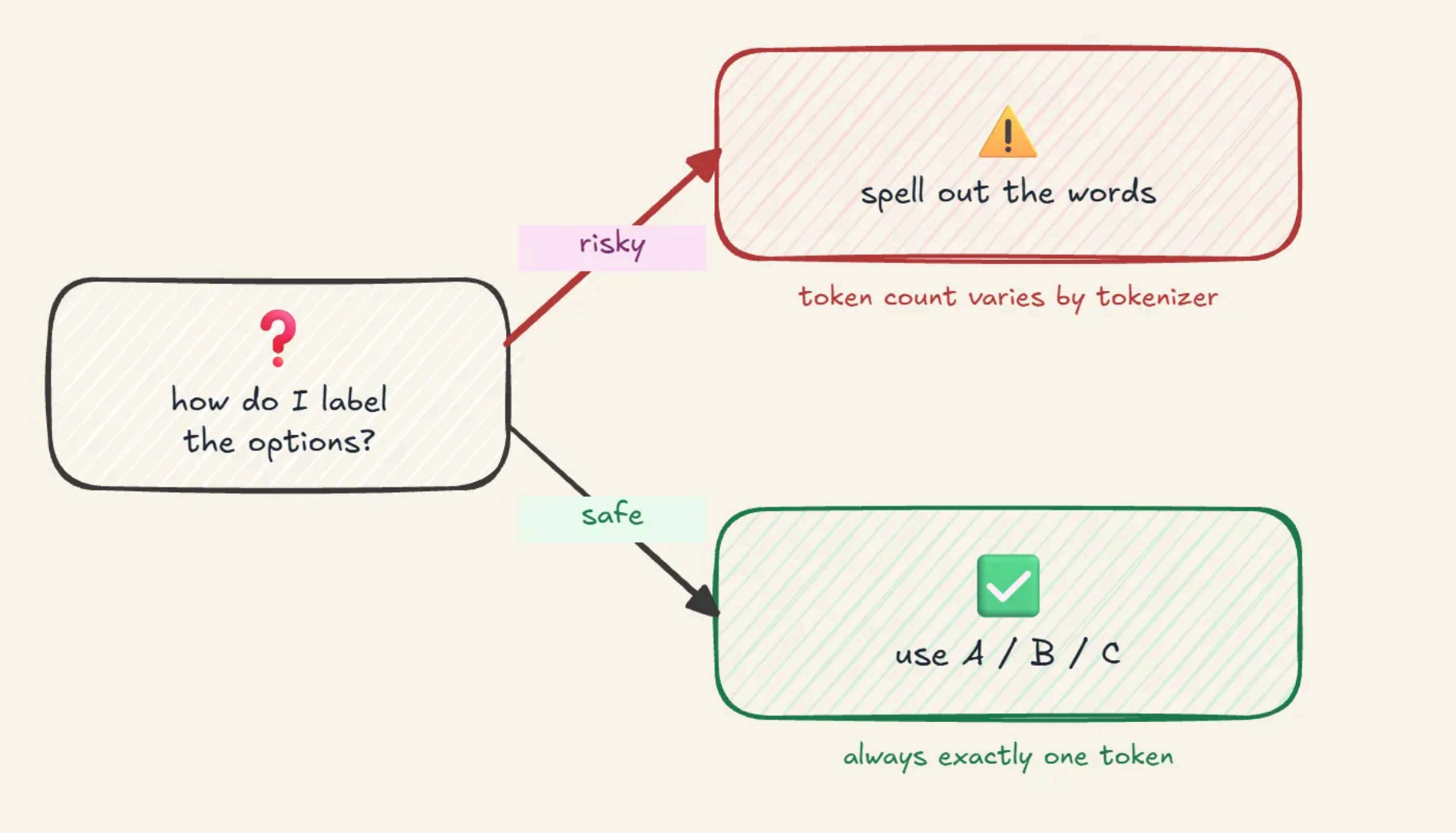
Walking through those same five checkpoints one at a time makes it easier to see exactly where the rest of the vocabulary gets thrown away.



Vocabulary logits get filtered down to the three declared token IDs, and only those three ever reach the restricted softmax on the way to a decision (Created by Fareed Khan)

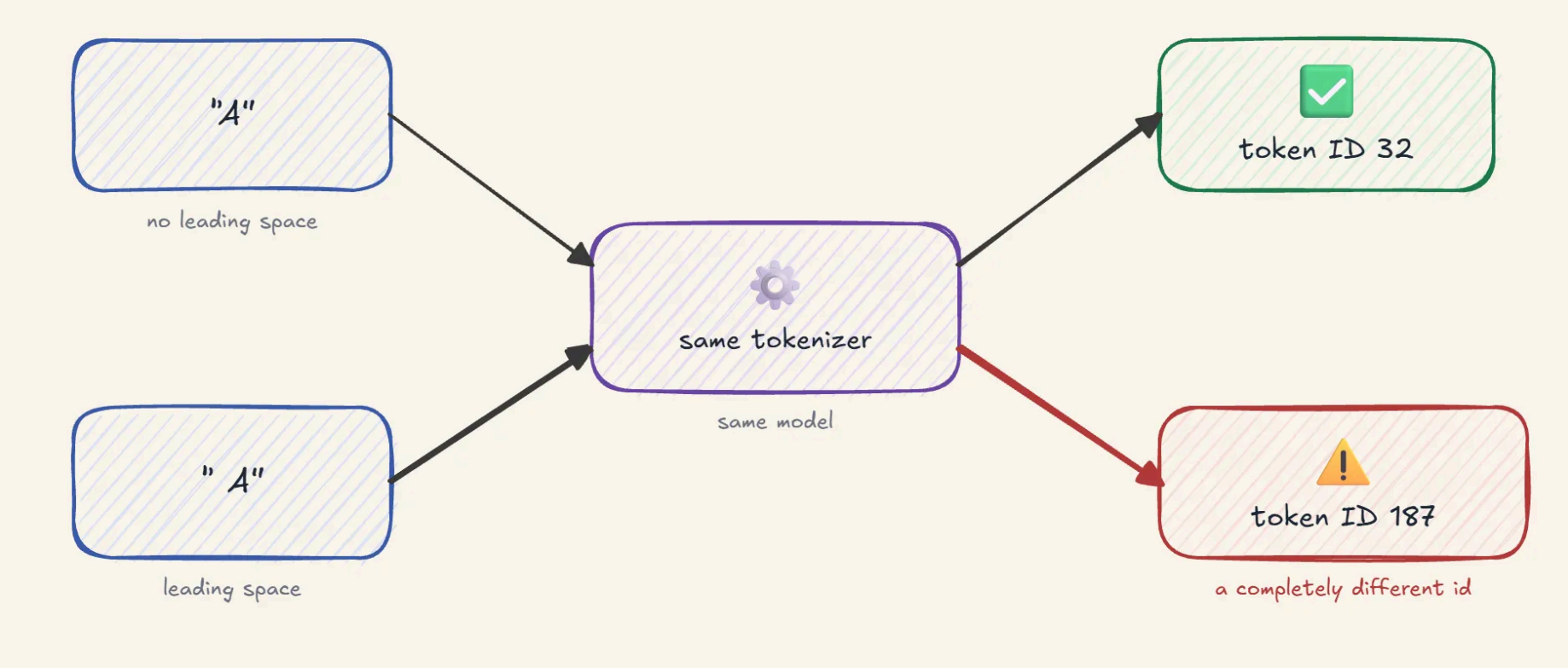
There is a real reason the labels are single letters, not full words. A word is not necessarily one token, `billing` might be one token for one tokenizer and several for another, and `technical support will definitely span more than one position.`

A single-letter label sits at exactly one output position no matter which words I care about, the real meaning still lives in the prompt next to each letter.



Spelling out a word risks a token count that shifts with the tokenizer, a single letter never does (Created by Fareed Khan)

There is also a small trap worth knowing first. Tokenizers often fold a leading space into a token, so `A` and `␣A` can map to two entirely different token IDs, and a chat template can insert that whitespace without me noticing.



The same letter, one with a leading space, resolves to a completely different token ID under the identical tokenizer
(Created by Fareed Khan)

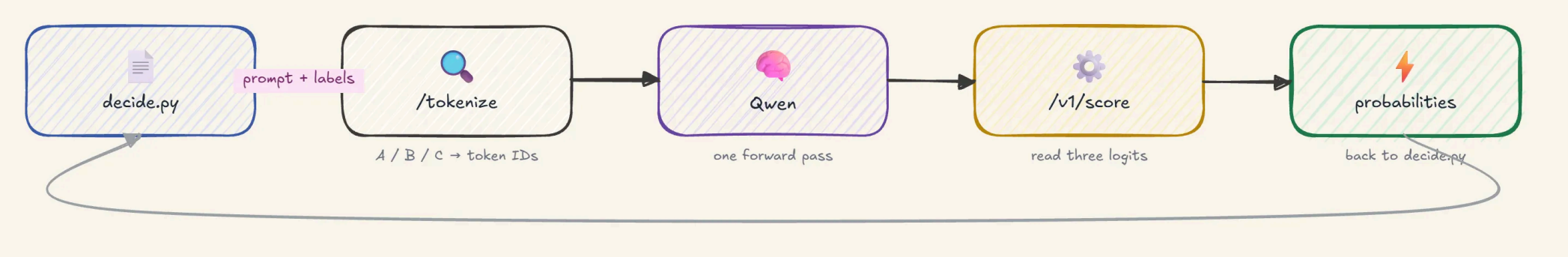
The fix is to render the whole prompt through the model's own chat template, check the exact continuation it expects, and verify each label really is one token before trusting a scoring call against it.

One more thing worth building in from day one. If the choices I hand the model are not exhaustive, restricted softmax will still force all the probability mass onto whichever declared answer is closest, even if none of them are right.

A security incident landing on a router that only knows `billing`, `technical`, `account` still gets confidently filed as one of those three, so adding an `OTHER` or `ESCALATE` option is cheap insurance.

Let's Build the Same Trick Ourselves

Reading about the mechanism only gets me so far, so let's run it. Jev itself is closed, but this exact scoring trick already sits inside open serving stacks, so I can reproduce it on a model I fully control. I am going to use SGLang as the server and Qwen2.5–0.5B-Instruct as the model, and build the client one small piece at a time.



decide.py talks to SGLang directly, one forward pass, three probabilities back (Created by Fareed Khan)

Starting the Model Server

First, a clean environment with the two packages I need.

```
python3 -m venv .venv
source .venv/bin/activate
pip install "sglang[all]==0.5.10.post1" "requests==2.34.2"
```

Then the model server itself, the only process doing any GPU work, the client script below just talks to it over HTTP.

```
python -m sglang.launch_server \
    --model-path Qwen/Qwen2.5-0.5B-Instruct \
```

```
    --host 127.0.0.1 \
    --port 30000
```

The first launch pulls the model down from Hugging Face, then it stays in memory on port 30000 for every request that follows.

Defining the Choices and Building the Prompt

Now the client, `decide.py`, built up one block at a time. First the labels and the ticket I am routing.

```
# decide.py
import json
import requests

BASE_URL = "http://127.0.0.1:30000"
MODEL = "Qwen/Qwen2.5-0.5B-Instruct"

# A is the token I actually score, "billing and payments" is what my
# application does with the answer once it comes back
choices = {
    "A": "billing and payments",
    "B": "technical support",
    "C": "account access",
}
ticket = "I was charged twice for the same subscription."
```

The dictionary order has to survive unchanged through tokenizing, scoring, and mapping the result back, so I build the prompt straight from it rather than typing the labels twice.

```
# decide.py (continued)
choice_lines = "\n".join(
    f"{label} = {meaning}" for label, meaning in choices.items()
)
prompt = f"""Ticket:
{ticket}
Question:
Which category matches the ticket?
Allowed labels:
{choice_lines}
Return only the label.
Label:
"""
print(prompt)
```

```
#### OUTPUT ####
Ticket:
I was charged twice for the same subscription.
Question:
Which category matches the ticket?
Allowed labels:
A = billing and payments
B = technical support
C = account access
```

```
Return only the label.
Label:
```

The prompt ends right at `Label:`, the exact position whose vocabulary vector I am about to read. I never ask SGLang to generate the token that would normally follow it.

Resolving the Label Token IDs

Before I can score anything I need the real token ID behind each of `A`, `B`, `C`, as this tokenizer sees them, not assumed from some other model's tokenizer.

```
# decide.py (continued)
label_token_ids = []

for label in choices:
    response = requests.post(
        f"{BASE_URL}/tokenize",
        json={
            "model": MODEL,
            "prompt": label,
            "add_special_tokens": False,
        },
        timeout=30,
    )
    response.raise_for_status()
    token_ids = response.json()["tokens"]
```

```
    if len(token_ids) != 1:
        raise ValueError(f"{label!r} is not a single token: {token_ids}")
    print(f"{label!r} -> {token_ids}")
    label_token_ids.append(token_ids[0])
```

```
#### OUTPUT ####
'A' -> [32]
'B' -> [33]
'C' -> [34]
```

Each label comes back as a single integer, so every option occupies exactly one vocabulary position. If any label had come back as two or more IDs, the `raise ValueError` above stops me right there.

Scoring the Three Positions

With the token IDs confirmed, the decision itself is one HTTP call.

```
# decide.py (continued)
response = requests.post(
    f"{BASE_URL}/v1/score",
    json={
        "model": MODEL,
        "query": prompt,
        "items": [""],
```

```
        "label_token_ids": label_token_ids,
        "apply_softmax": True,
    },
    timeout=120,
)

response.raise_for_status()
score_response = response.json()

print(json.dumps(score_response, indent=2))

scores = score_response["scores"][0]
```

`query` carries the whole prompt. The empty string in `items` tells SGLang to score the position right after it, `label_token_ids` names the three positions to read, and `apply_softmax` normalizes those readings into a proper probability distribution.

```
#### OUTPUT ####
{
  "scores": [
    [
      0.67776233,
      0.310878605,
      0.011359035
    ]
```

```
    ]
}
```

SGLang tokenized the prompt, ran Qwen once, and handed back three numbers, in the same order I passed the token IDs. No token was ever generated to produce this.

The raw logits behind that call, before `apply_softmax` normalized them, were 25.2776, 24.4982, and 21.1888 for `A`, `B`, `C`. Plugging those into the restricted softmax from the mechanism section, by hand, lands on the exact same 0.678 SGLang just printed.

$$P(A) = \frac{e^{25.2776}}{e^{25.2776} + e^{24.4982} + e^{21.1888}} \approx 0.6778$$

The real logits behind the real 0.678 this exact prompt produced against Qwen2.5-0.5B-Instruct

The exact real logits from this run, plugged into the restricted softmax by hand, landing on the same 0.678 (Created by Fareed Khan)

That computation is the same formula from earlier, run once on three numbers instead of tens of thousands.

Mapping the Scores Back to a Decision

The last step is plain Python, turning three floats back into the labels my application understands.

```
# decide.py (continued)
probabilities = {
    choices[label]: float(score)
    for label, score in zip(choices, scores, strict=True)
}

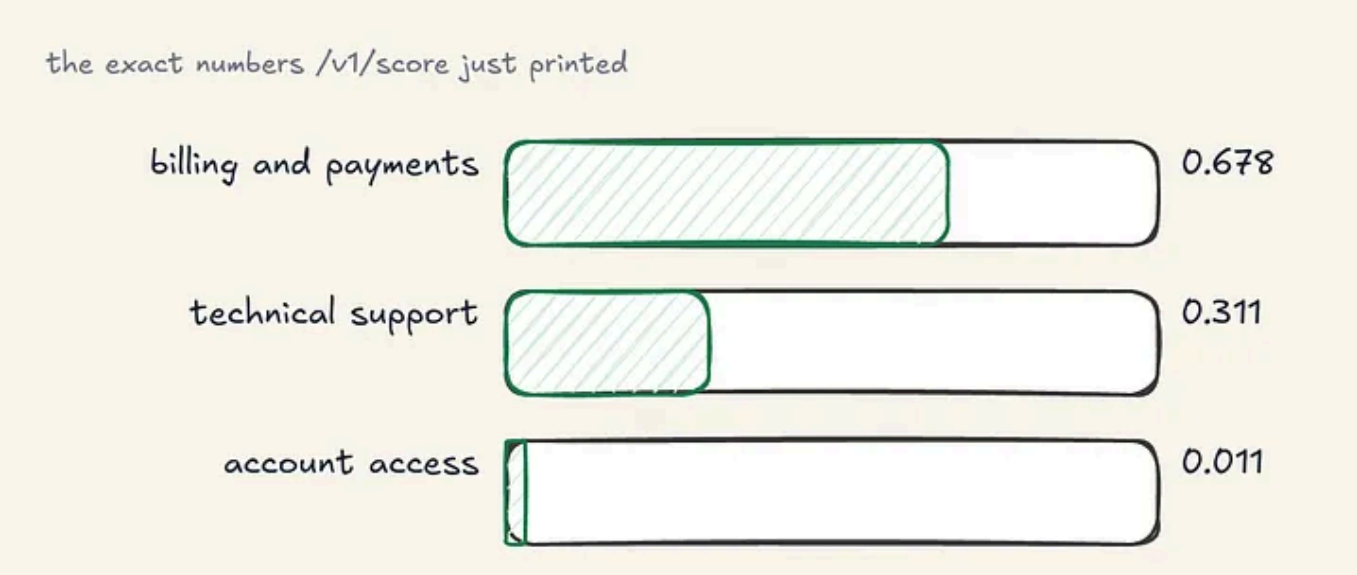
decision = max(probabilities, key=probabilities.get)

print(json.dumps({"decision": decision, "probabilities": probabilities}, indent=
```

```
#### OUTPUT ####
{
  "decision": "billing and payments",
  "probabilities": {
    "billing and payments": 0.6777623295783997,
    "technical support": 0.31087860465049744,
    "account access": 0.011359035037457943
```

```
    }
}
```

Seeing those three floats side by side as bars makes the gap obvious in a way the raw JSON does not.



The exact numbers /v1/score printed, drawn as bars, billing and payments ahead but not by an overwhelming margin (Created by Fareed Khan)

Billing wins, but only at **0.678**, nowhere near the 0.99-plus I might have expected, exactly the kind of case a threshold of 0.70 should catch and send for a second look rather than automate blindly.

It is worth measuring what this buys over generating an answer the normal way. I ran the Jev-style scoring path against the standard chat-completion path on the same SGLang server, same model, same 100 support-ticket cases, started at the same instant.

At the moment scoring finished all 100 cases, standard generation, which decodes up to 32 tokens per answer, had completed 54. Averaged out, scoring took **463 milliseconds** a case against **848 milliseconds** for generation, a measured **1.83x** gap, on identical hardware, same requests, same time.

Why the Number Next to the Answer Matters More Than the Answer

A typed answer only solves half the problem. The label `billing` tells me what won, the probability tells me how close the race was, and that second number is what my code should branch on.

```
{
  "choice": "billing",
  "probabilities": {"billing": 0.52, "technical": 0.46, "account": 0.02},
  "confidence": 0.06
}
```

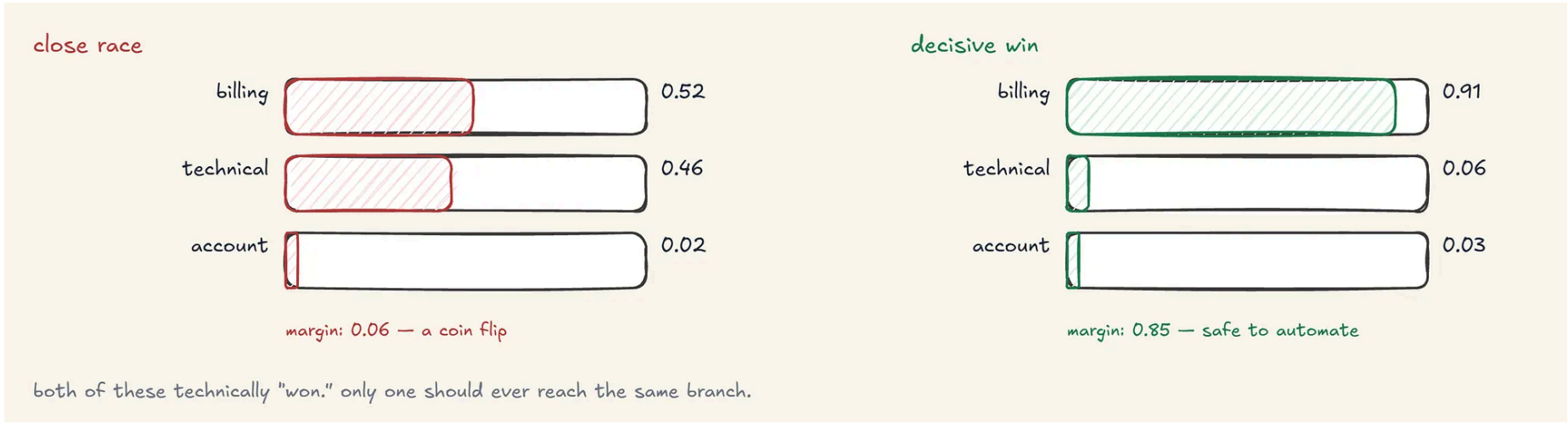
Automatically routing that ticket would be reckless, billing won but barely, and a low-confidence win should never take the same branch as a decisive one. One simple way to define confidence mathematically is the margin between the top answer and its closest rival, not the winning probability by itself.

$$\text{confidence} = p_{(1)} - p_{(2)}$$

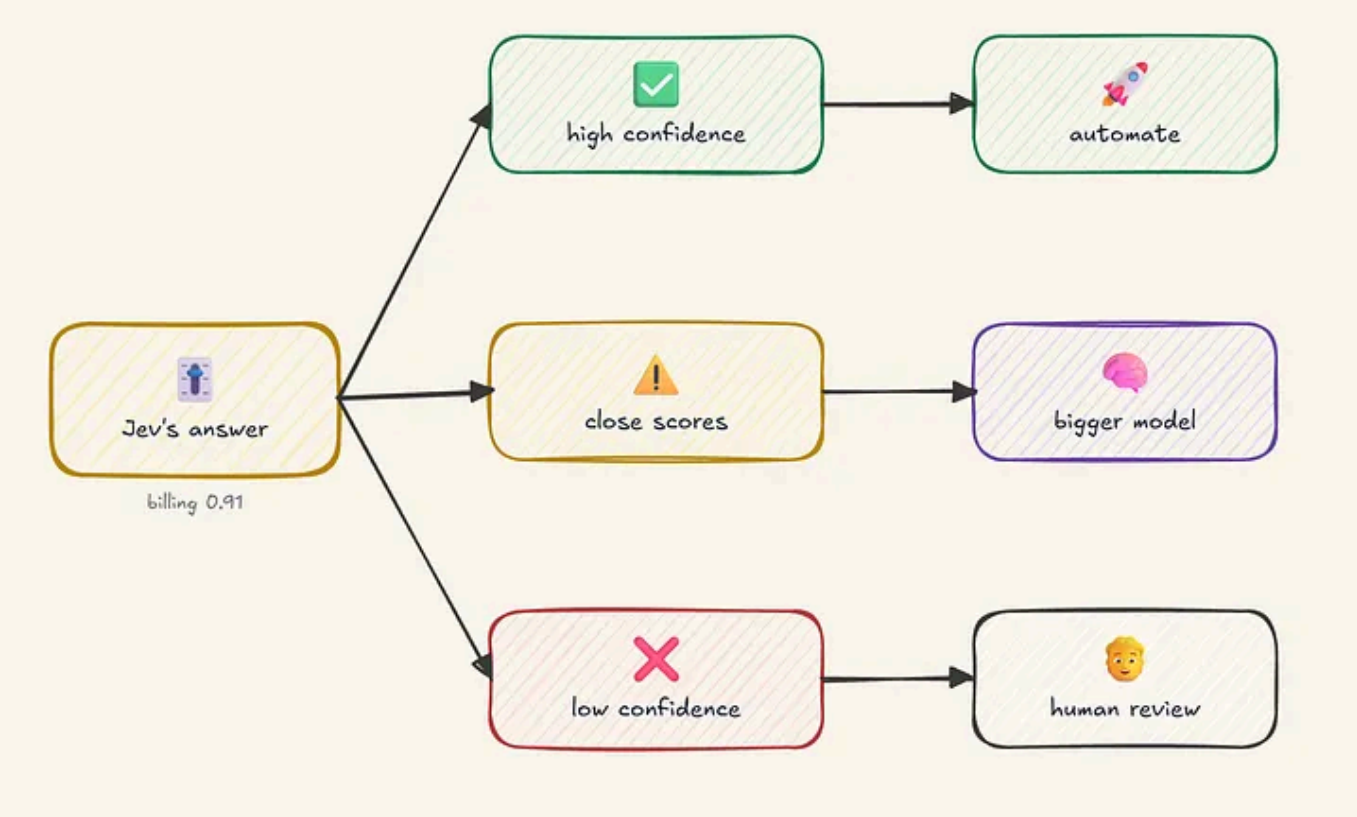
One honest way to read confidence, the gap between the winning answer and its closest rival

Confidence as a margin, the gap between the best answer and the one right behind it (Created by Fareed Khan)

Billing at 0.52 against technical at 0.46 gives a margin of only 0.06, practically a coin flip. A 0.52 winner and a 0.91 winner should never take the same branch even though both technically “won.”



The same headline, billing wins, hides two very different bar shapes underneath (Created by Fareed Khan)



High confidence automates, close scores escalate to a bigger model, low confidence goes to a human (Created by Fareed Khan)

```
if probabilities["billing"] > 0.9:
    route_automatically(ticket, team="billing")
elif confidence < 0.3:
    send_to_human_review(ticket)
```

```
    else:
        escalate_to_larger_model(ticket)
```

Stripped down, that whole if/elif block is one small rule with exactly two knobs to turn.

$$\text{action} = \text{automate if } p_{(1)} > \tau_{\text{hi}}, \quad \text{escalate if conf} < \tau_{\text{lo}}, \quad \text{else human}$$

The if/elif code above, written as one rule, two thresholds and nothing else

The routing code above, reduced to one rule with two thresholds (Created by Fareed Khan)

The thresholds live in code, where they can be reviewed, tested, and changed, not buried inside a model's weights. A dashboard label might tolerate a shaky prediction, a command that deletes customer data should demand a much higher bar.

TypeSafe trains Jev with what it calls Reinforcement Learning for Calibrated Decisions, RLCD for short. The goal is calibration, not just accuracy, a batch

of answers each at 90 percent confidence should turn out correct roughly 90 percent of the time. Bin every prediction by its stated confidence, then compare each bin's average confidence against how often that bin was right.

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left| \mathrm{acc}(b) - \mathrm{conf}(b) \right|$$

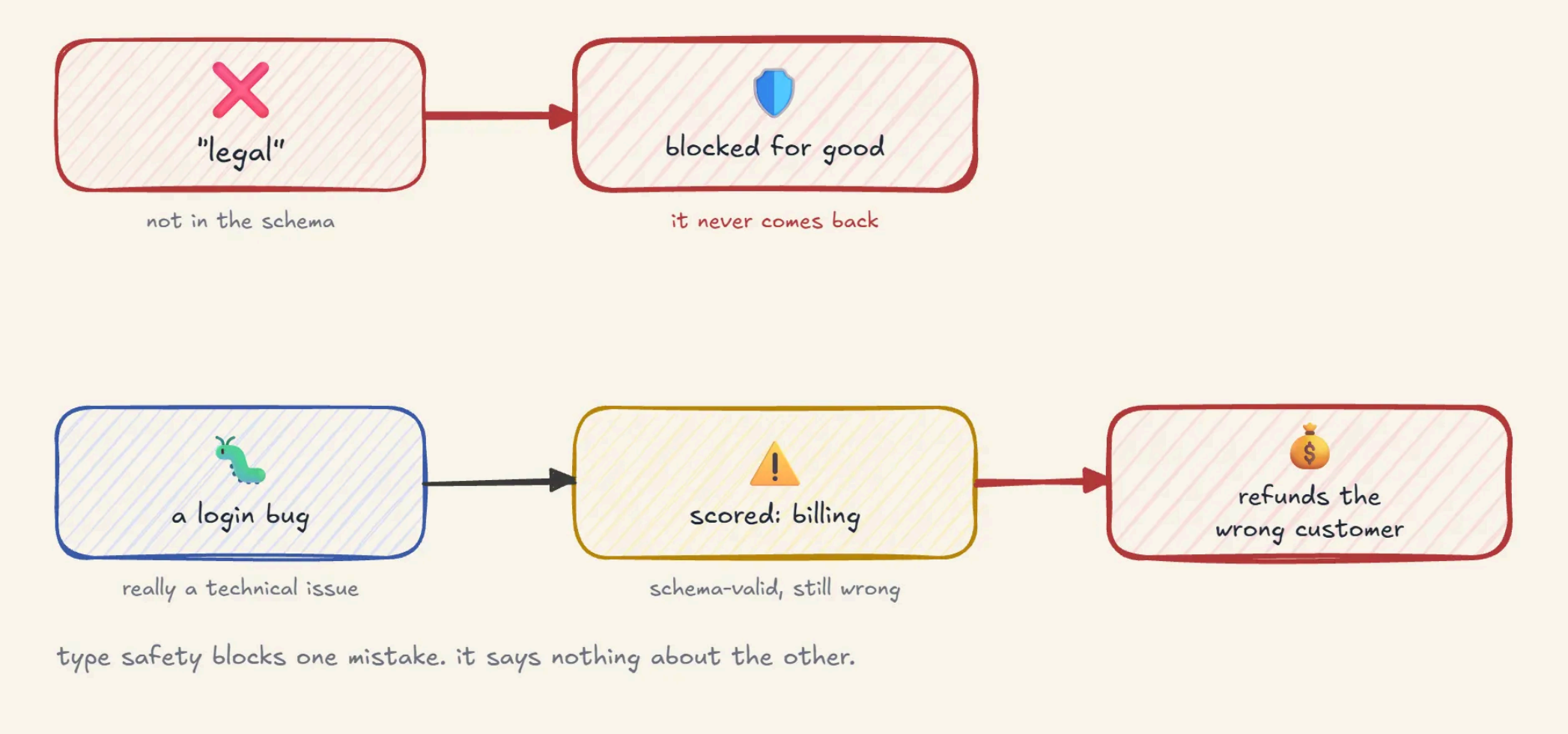
Bin every prediction by its stated confidence, then check how often that bin was actually right

Expected calibration error, binning predictions by confidence and checking each bin against its own accuracy
(Created by Fareed Khan)

A model can be accurate on average while still being miscalibrated, confidently wrong exactly in the band I would otherwise trust most. Confidence that does not track real correctness is decoration, not a signal I can threshold on, and checking it needs a labeled evaluation set of my own.

The Hallucination Claim, Said More Carefully

TypeSafe says Jev cannot hallucinate. That is true, but only under a narrow definition.



A schema blocks an answer outside its list, it says nothing about a wrong answer inside the list (Created by Fareed Khan)

If I declare `billing, technical`, `account` as the only valid answers, Jev's response can never come back as `legal`, that option simply does not exist in the schema. It also cannot hand me malformed prose. That part of the hallucination problem really is gone.

But Jev can still confidently pick the wrong valid option. A login bug scored as `billing instead of technical` is schema-valid, still wrong, and now somebody refunds the wrong customer over an issue that was never about a charge. The schema only rules out an answer that does not exist, it says nothing about whether the answer that does exist is right.

A more careful sentence than "Jev cannot hallucinate" is that it cannot break its declared output schema, but it can still be wrong. Type safety and correct judgment are two different guarantees, only one comes for free.

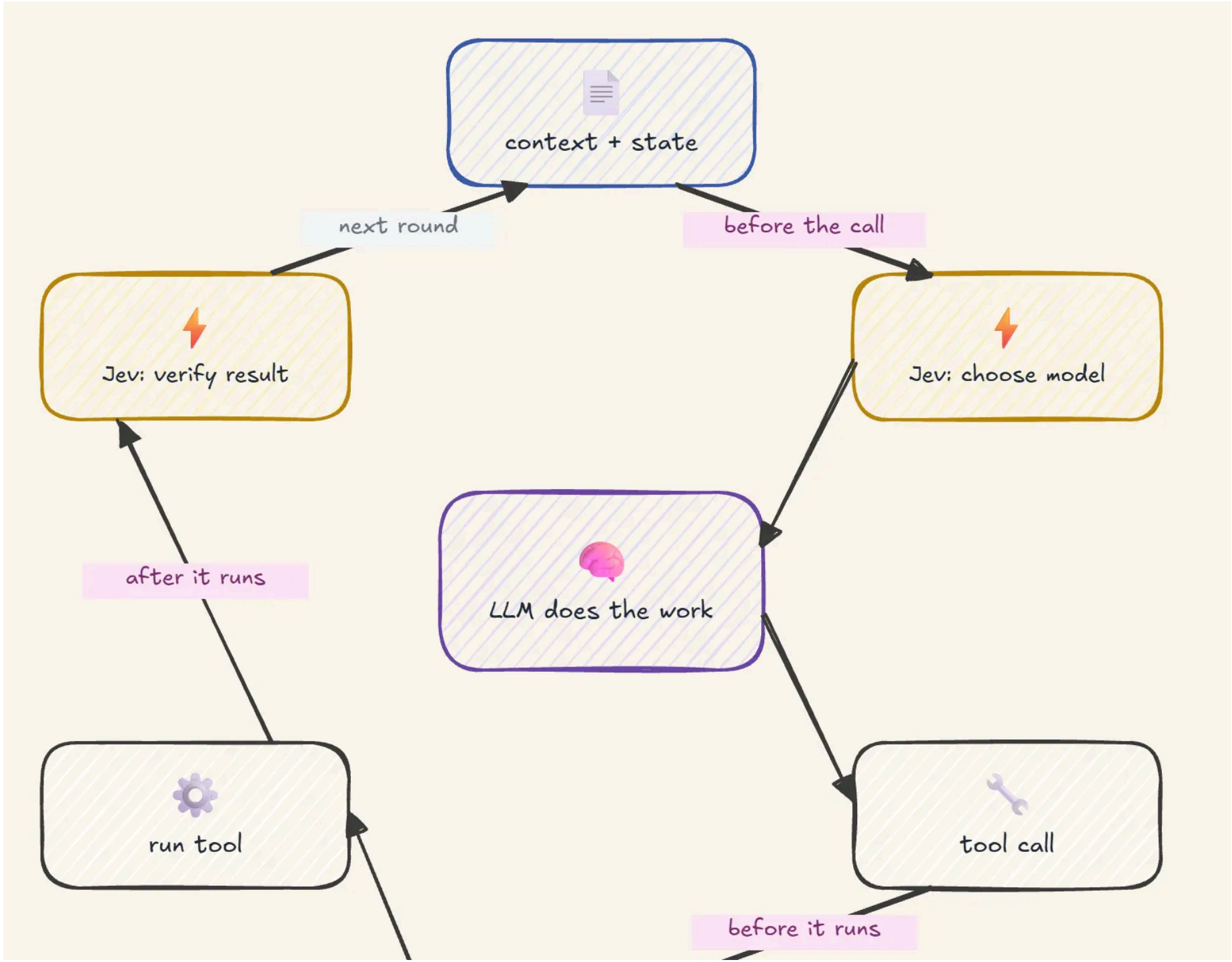
$$P(\text{answer} \in \mathcal{S}) = 1\,, \qquad P(\text{answer} = \text{true label}) < 1$$

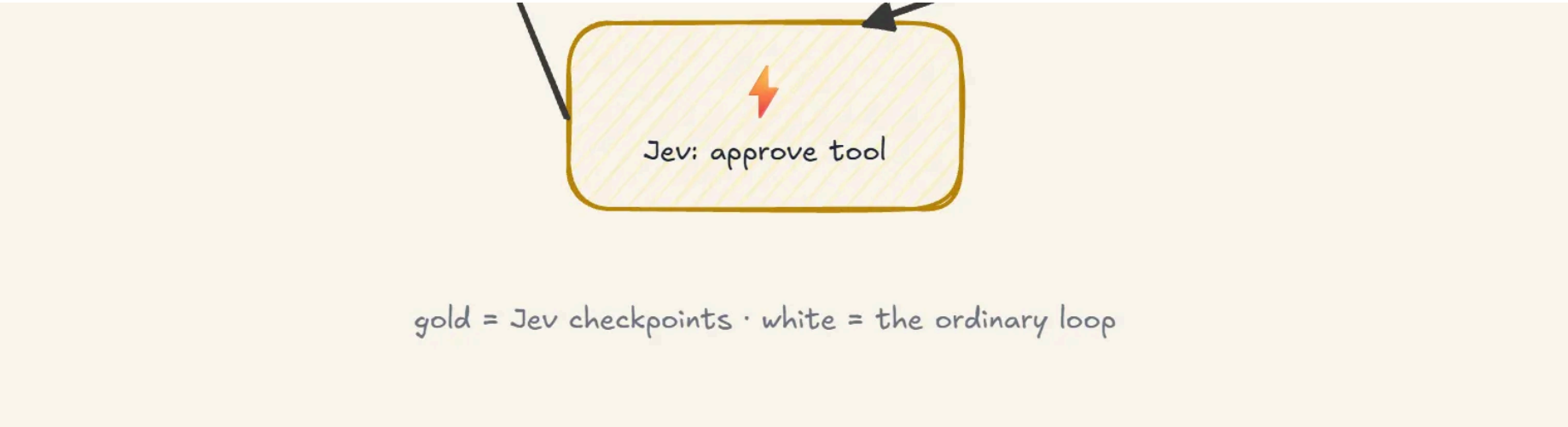
Schema membership is guaranteed at 1, correctness never is, and those are two different promises

Jev checkpoints sit beside the ordinary agent loop at three points, choosing the model, approving the tool, and verifying the result (Created by Fareed Khan)

Where Jev Earns Its Place Inside an Agent

Jev works best alongside a generative model, not instead of one. The generative model plans, writes, explains, and calls tools, the work that genuinely needs language. Jev handles the frequent small decisions around that work.

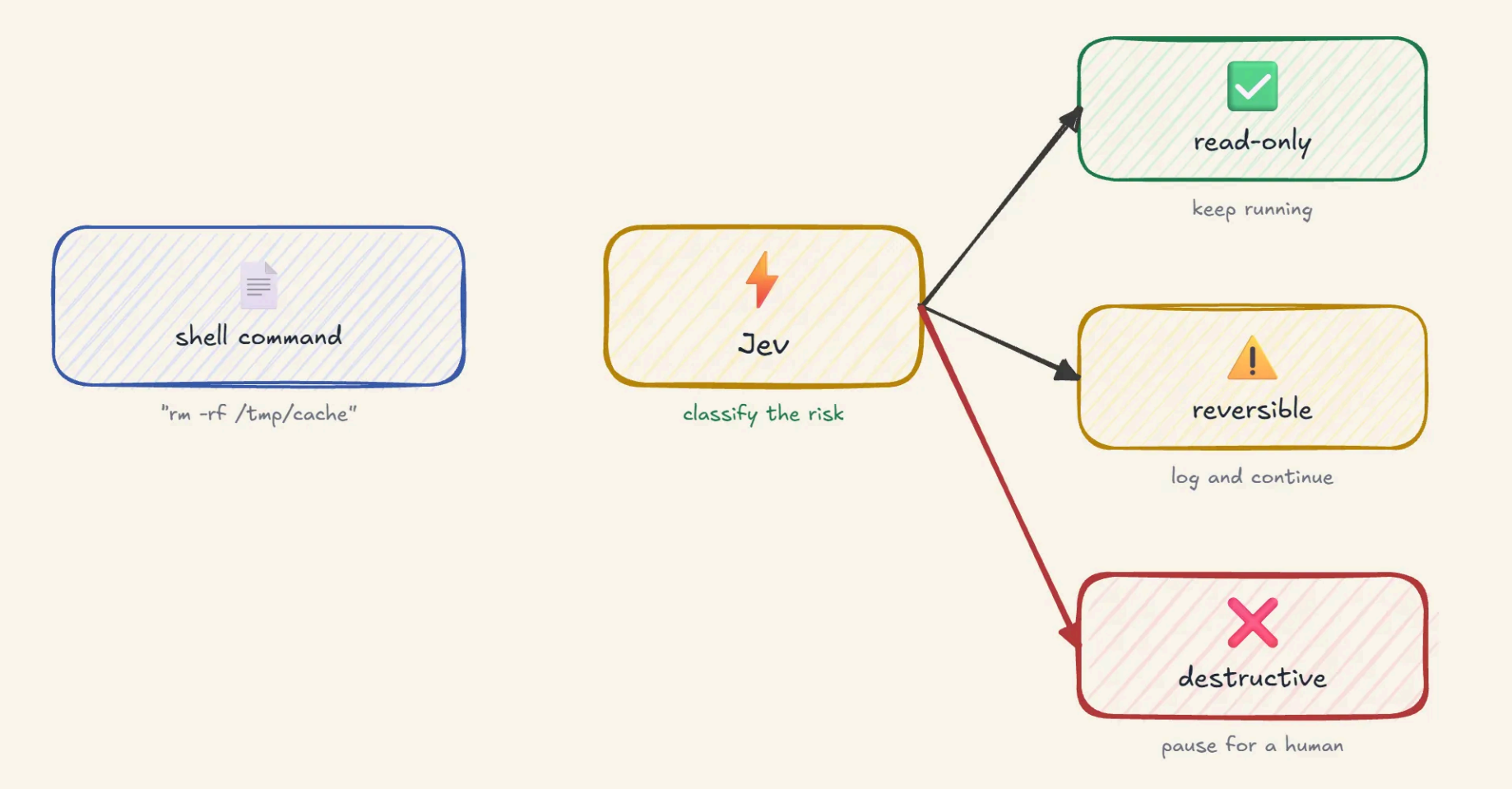

context + state
next round
before the call
Jev: verify result
Jev: choose model
after it runs
LLM does the work
run tool
tool call
before it runs



Membership in the declared schema is guaranteed at probability 1, correctness never is (Created by Fareed Khan)

Model routing. A simple lookup does not need the same model as a hard architecture review. Jev scores the incoming request and picks the least expensive model likely to finish the job, the router decides which model should answer, not the answer itself.

Tool risk gating. Before an agent runs a shell command, Jev can classify it as read-only, reversible, or destructive, alongside whether it touches production or leaves the sandbox. A high-confidence read-only command keeps running, a destructive or uncertain one pauses for a human.



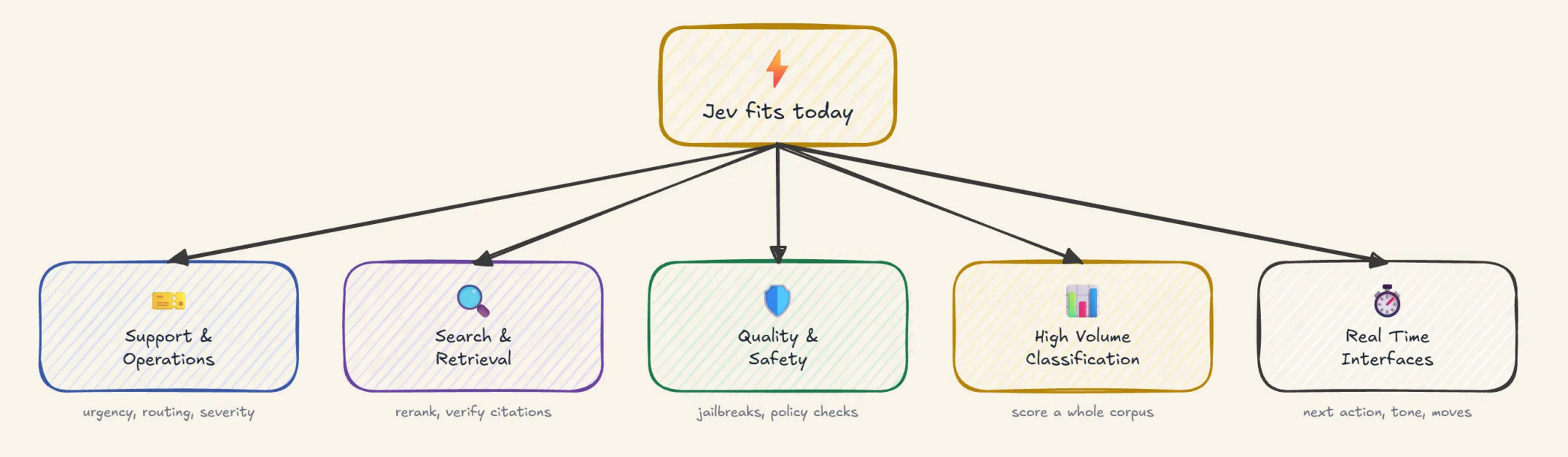
One shell command, one Choice call, three possible risk classes, three different consequences (Created by Fareed Khan)

Verification. An agent can claim a task is finished while its tests are still failing. Jev inspects the state and answers bounded questions, did the tests

pass, is the agent repeating the same failed action, does the output follow policy. A hard test still catches what it is built to catch, Jev adds a semantic check for what that hard rule cannot cover.

Problems Jev Can Solve Today

The workloads where Jev genuinely helps share three properties. I can name every possible answer ahead of time, a careful human could judge the input quickly by hand, and the decision happens often enough that latency or cost start to matter.



Five places the same typed-decision trick keeps paying off (Created by Fareed Khan)

- **Support and operations.** Classifying intent, urgency, department, spam, and frustration in one pass. Routing refunds through several small checks instead of one giant prompt. Ranking incidents by severity before a human reads them.
- **Search and retrieval.** Reranking passages by whether they answer the query, not just semantic closeness. Checking whether a citation genuinely supports a claim. Filtering irrelevant chunks before they reach an expensive model.
- **Quality and safety.** Screening prompts for jailbreak attempts or injection. Checking generated content against a written policy. Flagging a risky code change or tool call before it executes, beside deterministic checks, not replacing them.
- **High volume classification.** Labeling documents or customer messages at a scale that was never affordable before. Turning free text into features for a traditional ML model. Scoring every row of a corpus against the same rubric.
- **Real time interfaces.** Choosing the next browser action from a known set of page elements. Scoring tone while someone is still typing. Jev is text-only today, so any of this needs the environment turned into text first.

Does Any of This Hold Up

TypeSafe's own numbers deserve skepticism, a company benchmarking its own product picks the comparison that flatters it. It reports 70 to 500 milliseconds latency, $0.042 per million input tokens with output free, and claims of roughly 200 times faster and 400 times cheaper than comparable workflows.

Treat that as a ceiling, not a promise, the mechanism is real, as I just measured above, but marketing multiples sit at the favorable end on purpose.

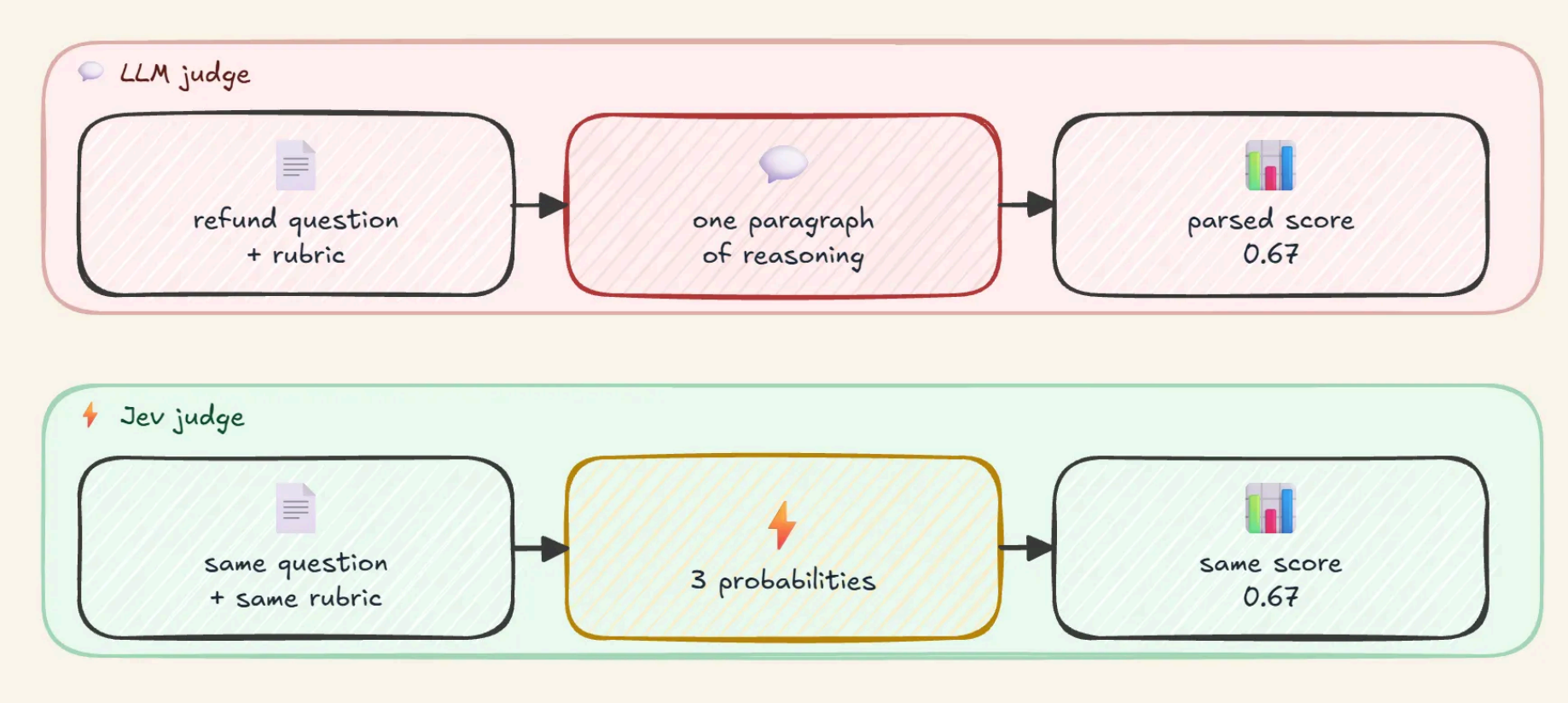
What I want instead is a comparison run by somebody with no stake in Jev looking good. LangChain ran exactly that kind of test, comparing Jev against three LLM judges, GPT-5.6 Luna, GPT-5.6 Terra, and Claude Sonnet 4.6, using their own Deep Agents harness and LangSmith for the dataset.

One example makes the mechanism difference vivid before the numbers do. Asked the same refund-policy question, an LLM judge reasons through a paragraph first, before producing a score of 0.67.

The answer gives a 30-day window, which matches policy, but leaves out the

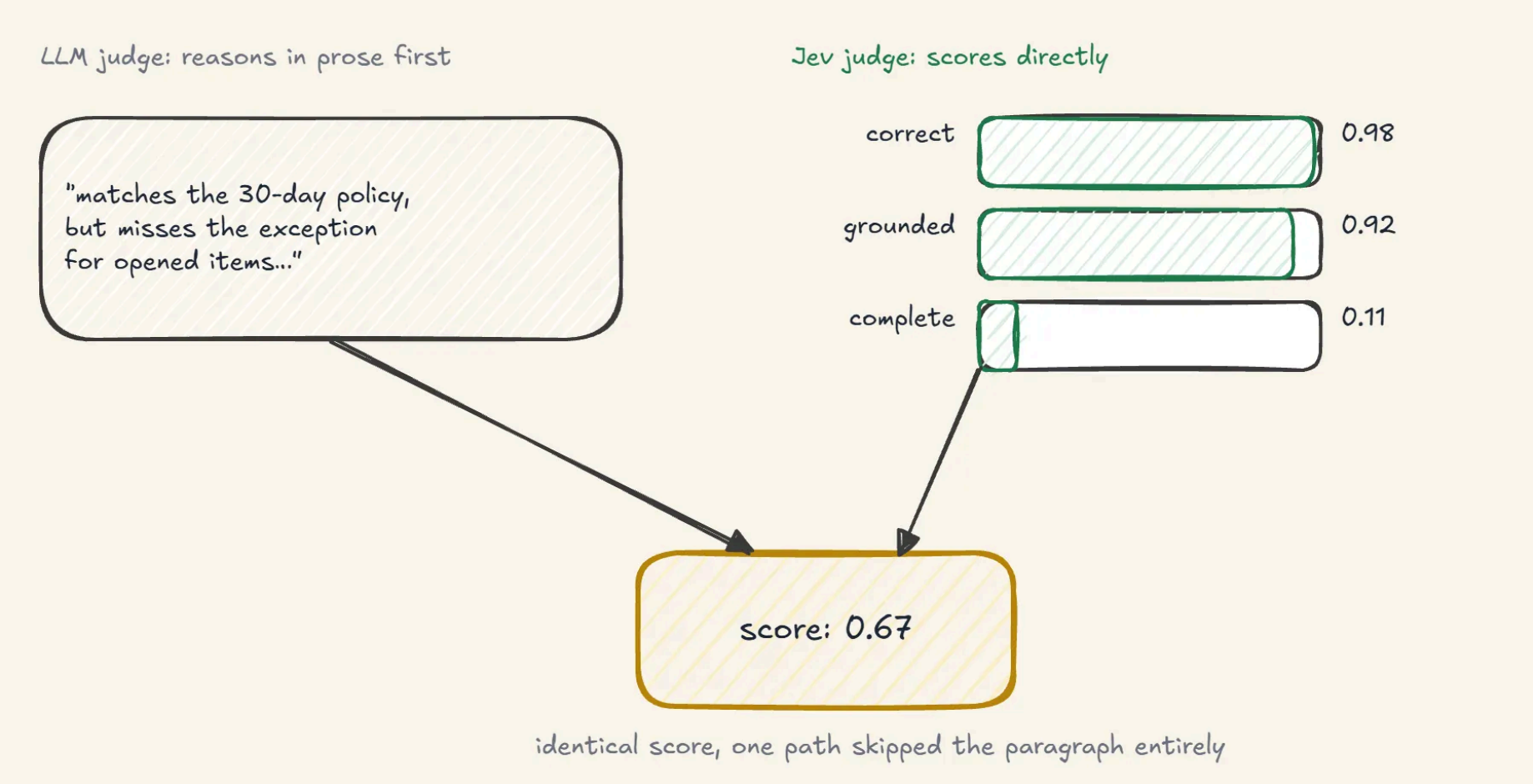
A Jev-style judge skips that paragraph entirely and returns three calibrated probabilities directly, correct 0.98, grounded 0.92, complete 0.11, which combine to the exact same 0.67.

exception for opened items, so it is accurate but not complete.



The same refund question, scored two ways, a paragraph of reasoning versus three direct probabilities, landing on the identical 0.67 (Created by Fareed Khan)

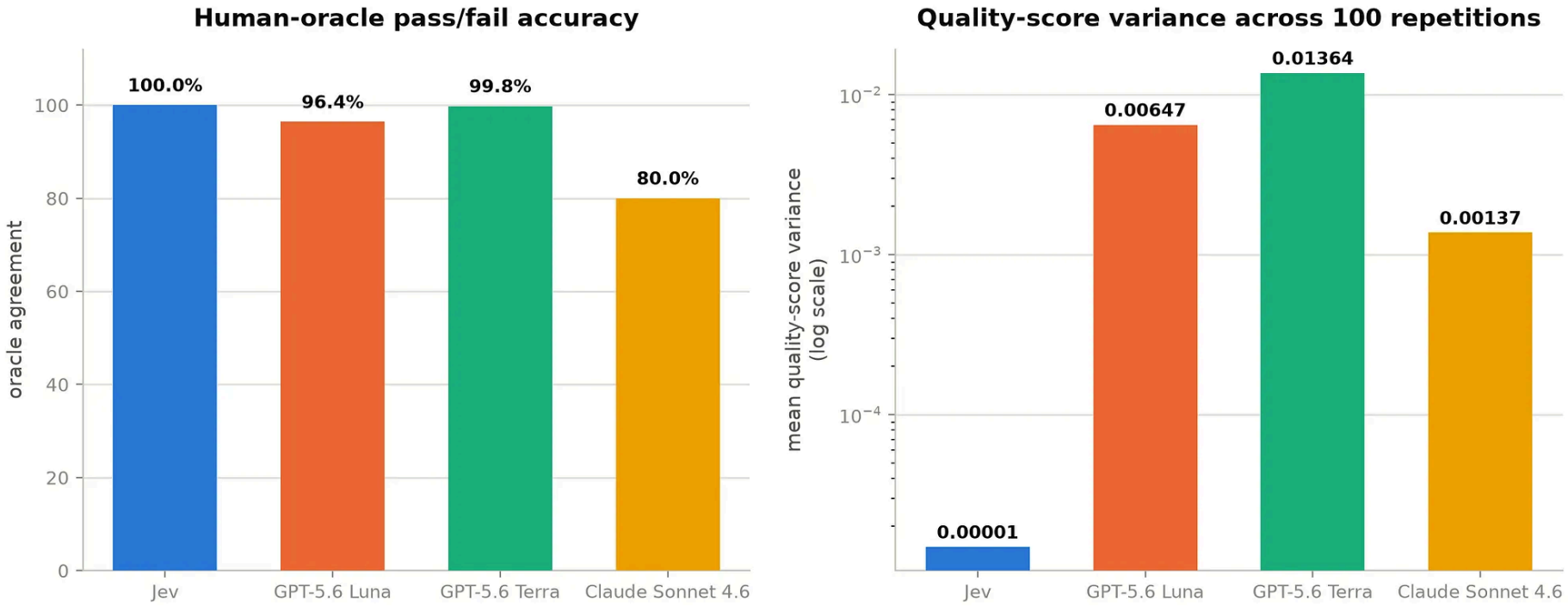
Drawn out as bars instead of icons, the same convergence is even clearer, one path writes a paragraph first, the other never does, and both arrive at 0.67.



The refund example again, this time as bars converging on the same 0.67 from a paragraph on one side and three direct probabilities on the other (Created by Fareed Khan)

Across 500 repeated pass/fail judgments against a human oracle, Jev matched the human on **100.0 percent** of them, GPT-5.6 Terra on 99.8

percent, GPT-5.6 Luna on 96.4 percent, and Claude Sonnet 4.6 on 80.0 percent.



Jev matches the human oracle on every repeated judgment, the three LLM judges do not (Created by Fareed Khan)

Accuracy alone tells half the story, a judge can be right on average while swinging wildly run to run on the same input. Measuring variance across 100 repetitions on a frozen case is where the real gap opens.

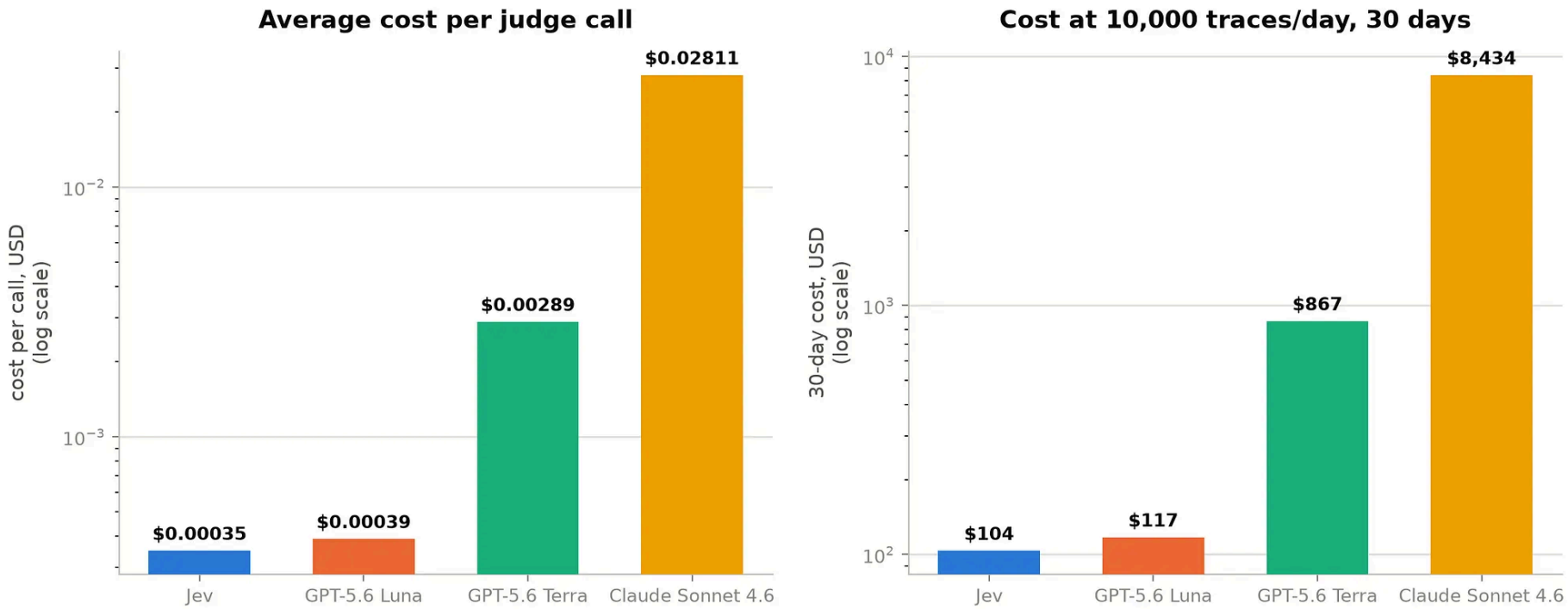
$$\mathrm{Var}(q) = \frac{1}{N}\sum_{n=1}^{N}(q_n - \bar{q})^2$$

How far 100 repeated scores on the same frozen case drift from their own mean

Variance across 100 repeated scores on one frozen, unchanged case (Created by Fareed Khan)

Jev's mean variance came out at 0.0000149. Claude Sonnet 4.6 was **92 times** higher, GPT-5.6 Luna **433 times** higher, and GPT-5.6 Terra **913 times** higher. Lower variance alone does not prove a judge is right, it can be consistently wrong, but once a judge is accurate, low variance is what makes that accuracy something to build automation on top of.

Cost and latency follow the same pattern. Jev averaged **$0.00035** and **0.44 seconds** per call, against $0.00039 and 2.50 seconds for GPT-5.6 Luna, $0.00289 and 2.83 seconds for GPT-5.6 Terra, and $0.02811 and 2.16 seconds for Claude Sonnet 4.6.



Jev costs a fraction of a cent per call, the gap widens sharply once results run at scale (Created by Fareed Khan)

Projected to 10,000 traces a day for 30 days, that gap becomes a genuinely different operating cost, **$103.59** for Jev against **$8,434.08** for Claude Sonnet 4.6, with GPT-5.6 Luna at $117.24 and GPT-5.6 Terra at $866.61.

LangChain also computed a signal value, oracle agreement multiplied by repeatability, the chance two independent calls on the same frozen trace

land on the same verdict.

$$\text{repeatability} = P(\text{verdict}_1 = \text{verdict}_2 \mid \text{same input})$$

The chance two independent calls on the identical frozen trace return the identical verdict

Repeatability, the probability two independent calls on the identical input agree with each other (Created by Fareed Khan)

Multiplying that by oracle agreement rewards a judge for being both correct and repeatable, and penalizes one that is confidently, consistently wrong.

$$\text{signal} = \mathbb{1}[\,\text{matches oracle}\,] \times \mathbb{1}[\,\text{two calls agree}\,]$$

Counted per repetition, a judge earns signal only when it is both correct and repeatable

Signal value counts a repetition only when the judge is both correct and repeatable, not one or the other (Created by Fareed Khan)

A judge right only sometimes, or repeatable only while wrong, earns nothing under that definition. Jev scored 100.0 percent signal value, GPT-5.6 Terra 99.4 percent, GPT-5.6 Luna 90.1 percent, and Claude Sonnet 4.6 80.0 percent.

That experiment used one agent, five test cases, and a narrow evaluation slice, so I would not treat it as the final word on every workload. But it is an independent measurement, not a vendor's own benchmark, and it points the same direction the mechanism itself predicts.

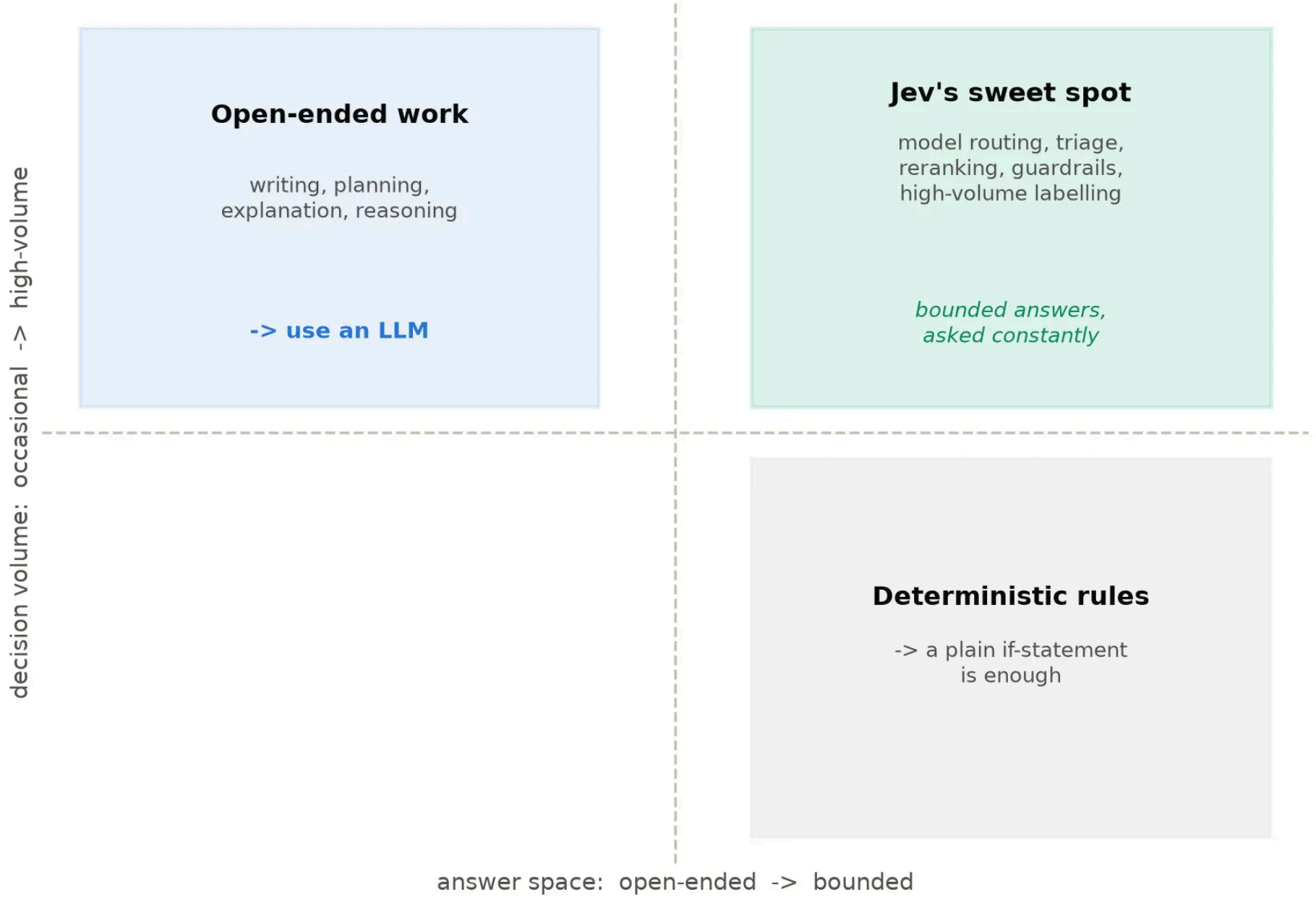
Where Jev Falls Apart

Jev gets less useful the moment the answer space stops being known ahead of time.

- It cannot write a response, summarize a document, generate code, or explain its own reasoning.
- It is unreliable at arithmetic, counting, date comparison, or exact string manipulation, keep those in ordinary code.
- It struggles when a decision needs several hidden reasoning steps chained together, split the judgment into smaller questions or use a reasoning model.

- It cannot extract an unknown value directly out of text, find the candidates first with something else, then let Jev choose among them.
- Irrelevant context can reduce its accuracy, send it only the state a decision needs.
- Closed weights, an early access window, text-only input, and still-limited independent calibration data mean it is too early to trust it blindly on a decision with real consequences.

Where a typed decision model actually fits



Bounded, high-volume answers are Jev's sweet spot, open-ended work still belongs to an LLM, and rare bounded decisions belong to a plain if-statement (Created by Fareed Khan)

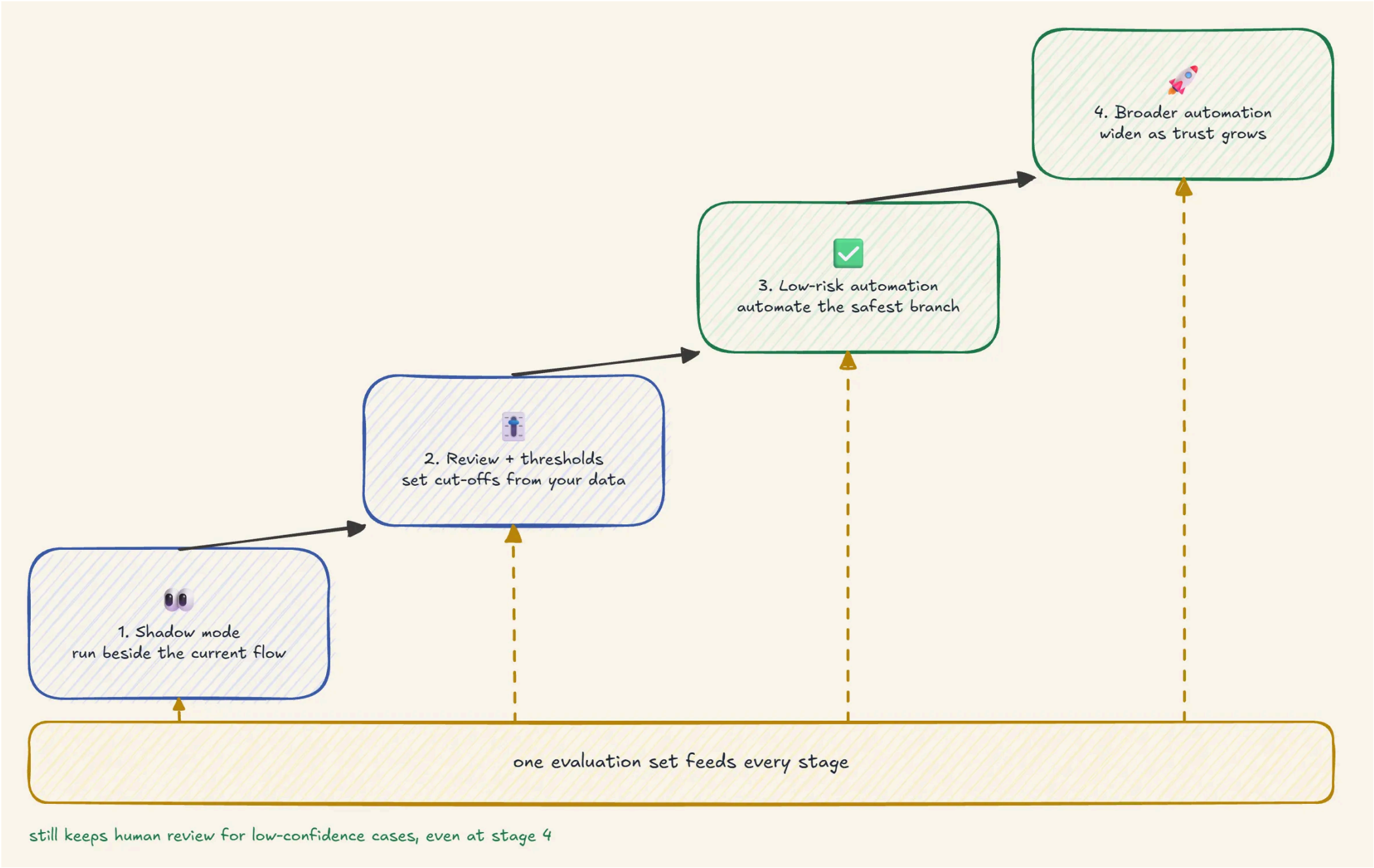
There is a simpler rule underneath all of this. If deterministic code already solves the problem correctly, keep the code, a plain `if` statement is faster, cheaper, and easier to test than any model.

A Sane Way to Roll This Out

A cheap model can still turn out expensive if its mistakes generate retries, manual review, or an incident, so the rollout has to measure the whole workflow, not just the token price.

1. **Choose one bounded, low-risk decision.** A small, clearly named set of possible answers, not the riskiest thing in the pipeline.
2. **Write the rubric before calling the model.** Decide in words what belongs in every option, the rubric is part of the system, not an afterthought.
3. **Collect representative examples first.** Include the ambiguous and adversarial cases on purpose.
4. **Run it in shadow mode.** Let it answer every real case without changing anything, and log its probabilities beside the current outcome.

5. **Plot accuracy against confidence and set thresholds from that data.** The cut-off should come out of the shadow-mode numbers, never a guess.
6. **Automate the safest branch first.** Keep a human, or a stronger model, as the fallback for anything the thresholds flag as uncertain.
7. **Version everything.** Pin the model version, the questions, the criteria, and the thresholds together, so a change can be replayed against the same evaluation set.



Four stages, one evaluation set feeding all of them, human review kept for anything uncertain even at the last stage
(Created by Fareed Khan)

Step five is doing real work. I want the highest bar that still automates enough volume to be worth building, not the highest bar in the abstract.

$$\tau^* = \arg\max_{\tau} \ \text{accuracy}(\tau) \ \text{ subject to } \ \text{coverage}(\tau) \geq c_{\min}$$

Pick the cut-off from shadow-mode data, the highest accuracy that still automates enough of the volume to matter

Pick the threshold that maximizes accuracy while still automating at least the minimum volume the business needs
(Created by Fareed Khan)

Raise the bar too far and the system never makes a mistake because it never decides anything, that is not automation, it is expensive logging. The coverage constraint keeps that honest.

What's Proven Here

The mechanism itself is proven, I built it and ran it above, with real token IDs and a real restricted softmax producing a real 0.678. The independent

evaluation numbers are also real, published by a party with nothing to gain from making Jev look better than it is.

What is not proven yet is longer:

- Jev's own weights are closed, so the mechanism I verified is a faithful reconstruction on an open model, not an inspection of Jev itself.
- TypeSafe's own latency and cost claims sit at the favorable edge of their own comparisons.
- Independent calibration data outside the one evaluation cited here is still thin.
- Access is early, and the interface is text-only, so an image, a UI, or raw sensor data needs a separate step to become text first.

None of that makes the underlying idea wrong. It makes the claims around it precise rather than settled.

Where This Leaves Us

Jev is not interesting because it beats a language model at writing. It is interesting because it refuses to write at all, fixed answer types, explicit

uncertainty on every answer, every question evaluated in parallel, and code that keeps the branching logic instead of handing it to the model.

That makes it a companion to generative models, not a replacement for them. The generative model still produces the plan, the explanation, or the code. Jev routes the request, gates the risky action, checks the result, and decides when uncertainty is high enough that a human needs to look.

If you build one thing from this post, do not start by rebuilding your whole agent around a typed decision model. Find one place your code pays for a full generative call just to get a one-word answer, give it the minimum state it needs, and let it earn one branch before you hand it anything more.

The mental model worth keeping is the simplest one, this kind of model adds judgment exactly where an ordinary `if` statement already understands the values but never understood what they meant.

Thanks for reading this far.

AI + | Artificial Intelligence + | Data Science ✓ | Machine Learning ✓

Python ✓



Written by Fareed Khan

77K followers · 0 following

Follow

I write on AI, https://www.linkedin.com/in/fareed-khan-dev/

Published in Level Up Coding

365K followers · Last published 1 day ago

Follow

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech